

RAPPORT DE PROJET — FILIÈRE DATA SCIENCE & BIG DATA (DSBD)

Assistant Juridique Intelligent

Conception, mise en œuvre et gouvernance d'une chaîne
MLOps pour un système RAG juridique marocain

Version 0.2.0 — Rapport final consolidé

Axes couverts par le projet

M1 — Ingestion & corpus juridique	M5 — API & couche de service
M2 — Chaîne RAG vérifiable	M6 — Frontend & expérience utilisateur
M3 — Suivi d'expériences & registre	M7 — Observabilité SRE / ML
M4 — DevOps & plateforme	M8 — Sécurité, gouvernance & conformité

Réalisé par — Groupe CloudMind

Youssef El Alem (*coordinateur*)

Douae Moussaoui

Imane Ibnchakroune

Amal El Guerdani

Salma El Ouarrate

Nouhaila Fadli

Oumaima Jeraidi

Taha Kachmar

Module M7 — Observabilité

Module M1 — Ingestion & corpus

Module M2 — Chaîne RAG

Module M3 — Suivi & registre

Module M4 — DevOps & plateforme

Module M5 — API & service

Module M6 — Frontend & UX

Module M8 — Sécurité & conformité

Encadré par

Pr. Mohammed AIT DAOUD

Dr. Taoufik AMZIL

Faculté des Sciences Ben M'Sik

Année universitaire 2025 – 2026

Remerciements

Au terme de ce projet, nous tenons à exprimer notre gratitude à toutes les personnes qui ont contribué, directement ou indirectement, à son aboutissement.

Nos remerciements s'adressent en premier lieu à nos encadrants, le Professeur Mohammed AIT DAOUD et le Docteur Taoufik AMZIL, dont les orientations méthodologiques et les remarques critiques nous ont amenés à distinguer rigoureusement ce qui relève de la démonstration technique de ce qui relève de la simple déclaration d'intention — une distinction qui structure l'intégralité de ce rapport.

Nous remercions également le corps professoral de la filière *Data Science & Big Data* de la Faculté des Sciences Ben M'Sik, pour la formation qui nous a permis d'aborder ce projet avec les fondements théoriques nécessaires, ainsi que les membres du jury pour l'attention portée à l'évaluation de ce travail.

Enfin, nous tenons à souligner que ce projet est le fruit d'un travail collectif. Chacun des huit membres du groupe CloudMind a porté la responsabilité d'un module, mais aucun module n'aurait abouti sans les revues croisées, les échanges de remarques et les arbitrages collectifs qui ont accompagné chaque contribution.

Le groupe CloudMind

Résumé

Ce rapport présente la conception et la mise en œuvre d'un **assistant juridique intelligent** fondé sur l'architecture *RAG* (*Retrieval-Augmented Generation*, ou génération augmentée par récupération), appliquée à un corpus juridique marocain bilingue français-arabe. Le projet ne se limite pas à la construction d'un prototype fonctionnel : il vise l'industrialisation complète de cette chaîne selon les pratiques **MLOps**, c'est-à-dire sa reproductibilité, son observabilité et sa conformité réglementaire.

Le système est structuré en huit modules complémentaires, articulés autour d'un chemin critique reliant l'ingestion du corpus, la chaîne de recherche et de génération, l'exposition par une interface de programmation sécurisée, et l'affichage dans une interface utilisateur bilingue. Quatre modules transverses assurent respectivement le suivi des expériences, l'automatisation du déploiement, la supervision en production et la gouvernance de la conformité.

Deux contraintes ont structuré l'ensemble des choix techniques : d'une part l'exigence de *provenance vérifiable*, toute réponse générée devant être rattachée à des citations retrouvables mot pour mot dans le corpus source ; d'autre part l'exigence de *conformité*, la loi marocaine 09-08 relative à la protection des données à caractère personnel s'appliquant à tout traitement de données réelles.

La version 0.2.0, objet de ce rapport, livre une chaîne fonctionnelle de bout en bout, couverte par des tests automatisés et une pile d'observabilité déployée. Les limites subsistantes — notamment l'absence de corpus réel validé et l'absence de jeu d'évaluation annoté par des experts juridiques — sont documentées avec leur justification technique, au même titre que les réalisations.

Mots-clés : MLOps, RAG, grands modèles de langage, recherche hybride, observabilité, conformité, loi 09-08, intégration continue, droit marocain.

Abstract

This report presents the design and implementation of an **intelligent legal assistant** based on the *Retrieval-Augmented Generation* (RAG) architecture, applied to a bilingual French-Arabic Moroccan legal corpus. The project is not limited to building a working prototype: it aims at the full industrialisation of this pipeline according to **MLOps** practices, namely reproducibility, observability and regulatory compliance.

The system is organised into eight complementary modules, built around a critical path linking corpus ingestion, the retrieval and generation pipeline, exposure through a secured application programming interface, and rendering in a bilingual user interface. Four cross-cutting modules respectively handle experiment tracking, deployment automation, production monitoring and compliance governance.

Two constraints shaped every technical decision: first, the requirement of *verifiable provenance*, whereby every generated answer must be traceable to citations found verbatim in the source corpus; second, the requirement of *compliance*, as Moroccan law 09-08 on the protection of personal data applies to any processing of real data.

Version 0.2.0, the subject of this report, delivers a functional end-to-end pipeline, covered by automated tests and a deployed observability stack. Remaining limitations — in particular the absence of a validated real corpus and of an expert-annotated evaluation set — are documented together with their technical justification, on an equal footing with the achievements.

Keywords: MLOps, RAG, large language models, hybrid search, observability, compliance, law 09-08, continuous integration, Moroccan law.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des figures	vii
Liste des tableaux	viii
Liste des abréviations	ix
1 Introduction générale	1
1.1 Contexte et problématique	1
1.1.1 Première exigence : la provenance vérifiable	1
1.1.2 Seconde exigence : la conformité réglementaire	1
1.2 Objectif du projet	2
1.3 Organisation du travail	2
1.4 Structure du rapport	2
2 État de l’art et fondements théoriques	4
2.1 Le MLOps : définition et cycle de vie	4
2.1.1 Origine du besoin	4
2.1.2 Le cycle de vie MLOps	4
2.2 Les grands modèles de langage et leurs limites	5
2.2.1 Principe	5
2.2.2 La limite structurelle : l’hallucination	5
2.3 La génération augmentée par récupération (RAG)	6
2.3.1 Principe général	6
2.3.2 Le découpage du corpus	6
2.3.3 Les plongements vectoriels	7
2.3.4 Recherche lexicale : BM25	7
2.3.5 Recherche dense	7
2.3.6 Pourquoi combiner les deux : la fusion RRF	7
2.4 RAG contre ajustement fin : justification d’un choix	8
2.5 Les bases de données vectorielles	8
2.6 L’observabilité : les trois piliers	9
2.7 Intégration et livraison continues	9
2.8 La dérive des données et des modèles	10
2.9 Étude de l’existant	10
3 Analyse et conception	12

3.1	Analyse des besoins fonctionnels	12
3.2	Analyse des besoins non fonctionnels	12
3.3	Vue fonctionnelle : diagramme de cas d'utilisation	13
3.4	Vue dynamique : diagramme de séquence	15
3.5	Vue dynamique : diagramme d'activité	15
3.6	Vue statique : diagramme de classes	18
3.7	Vue physique : diagramme de déploiement	18
3.7.1	La règle de sécurité fondamentale	18
3.7.2	La séparation des réseaux	21
3.8	Synthèse de la conception	21
4	Méthodologie et gestion de projet	22
4.1	Organisation de l'équipe	22
4.2	Méthodologie de développement	22
4.2.1	Le choix du GitHub Flow	22
4.2.2	Les garde-fous de gouvernance	23
4.3	Planification du projet	23
4.3.1	Découpage temporel	23
4.3.2	Lecture du planning	24
4.3.3	Rythme effectif des contributions	24
4.4	Enseignements méthodologiques	25
5	Architecture globale	26
5.1	Vue d'ensemble	26
5.2	Les trois interfaces du chemin critique	27
5.3	Pourquoi ce séquençement	27
5.4	Pile technologique par couche	28
6	Détail des modules	29
6.1	M1 — Ingestion et corpus juridique	29
6.2	M2 — Chaîne RAG vérifiable	30
6.3	M3 — Suivi d'expériences et registre de modèles	31
6.4	M4 — DevOps et ingénierie de plateforme	32
6.5	M5 — API et couche de service	33
6.6	M6 — Frontend et expérience utilisateur	34
6.7	M7 — Observabilité SRE et ML	35
6.8	M8 — Sécurité, gouvernance et conformité	36
7	Résultats et démonstration	39
7.1	Interface utilisateur	39
7.1.1	Tableau de bord	39
7.1.2	Consultation juridique	39
7.2	Supervision et observabilité	39
7.2.1	Tableau de bord Grafana	40
7.2.2	Règles d'alerte	40
7.2.3	Cibles supervisées	40

7.2.4	Traçage distribué	42
7.3	Synthèse des résultats obtenus	42
8	Décisions transverses et gouvernance	44
8.1	Carte des dépendances inter-modules	44
8.2	Le point de convergence : le module M4	44
8.3	Une décision de périmètre valant pour l'ensemble du projet	45
8.4	Bilan de la gouvernance des contributions	45
9	Conclusion générale et perspectives	46
9.1	Bilan du travail réalisé	46
9.2	Réponse à la problématique	46
9.3	Limites du travail	47
9.4	Perspectives	47
9.4.1	Priorités à court terme	47
9.4.2	Perspectives à moyen terme	48
9.5	Apport personnel et formation	48
	Bibliographie	49
A	Répartition détaillée des responsabilités	51
B	Glossaire des termes techniques	52
C	Reproduction de l'environnement	54

Table des figures

2.1	Cycle de vie MLOps et positionnement des modules	5
2.2	Architecture d'un système RAG	6
2.3	Les trois piliers de l'observabilité	9
3.1	Diagramme de cas d'utilisation	14
3.2	Diagramme de séquence du traitement d'une question	16
3.3	Diagramme d'activité de la chaîne RAG	17
3.4	Diagramme de classes de la chaîne RAG	19
3.5	Diagramme de déploiement	20
4.1	Organigramme de l'équipe CloudMind	22
4.2	Flux de contribution GitHub Flow	23
4.3	Diagramme de Gantt du projet	24
5.1	Diagramme de composants : chemin critique et modules transverses	26
7.1	Tableau de bord de l'application	39
7.2	Écran de consultation juridique	40
7.3	Tableau de bord Grafana — API et chaîne RAG	41
7.4	Règles d'alerte chargées dans Prometheus	41
7.5	Cibles supervisées par Prometheus	42
7.6	Trace distribuée dans Tempo	43

Liste des tableaux

2.1	Comparaison entre RAG et ajustement fin	8
2.2	Étude de l'existant	11
3.1	Besoins fonctionnels	12
3.2	Besoins non fonctionnels	13
4.1	Répartition des contributions par semaine	25
5.1	Interfaces du chemin critique	27
5.2	Pile technologique par couche	28
6.1	Périmètre du module M4	33
6.2	Résultats du banc de test d'anonymisation	37
7.1	Synthèse des indicateurs vérifiables	43
8.1	Carte des dépendances inter-modules	44
8.2	Mécanismes de gouvernance appliqués	45
9.1	Trajectoire de versions du projet	47
A.1	Répartition des responsabilités par module	51
C.1	Services de la pile d'observabilité	54

Liste des abréviations

Sigle	Signification
AI Act	<i>Artificial Intelligence Act</i> — règlement européen 2024/1689 encadrant les systèmes d'intelligence artificielle
API	<i>Application Programming Interface</i> — interface de programmation applicative
BM25	<i>Best Matching 25</i> — fonction de pondération pour la recherche lexicale probabiliste
CI/CD	<i>Continuous Integration / Continuous Delivery</i> — intégration et livraison continues
DMZ	<i>Demilitarized Zone</i> — zone réseau intermédiaire hébergeant les seuls services exposés à l'extérieur
DTO	<i>Data Transfer Object</i> — objet de transport de données entre deux couches applicatives
DVC	<i>Data Version Control</i> — outil de versionnage des données et des pipelines
GHCR	<i>GitHub Container Registry</i> — registre d'images conteneurisées
HMAC	<i>Hash-based Message Authentication Code</i> — code d'authentification fondé sur une fonction de hachage
HTTP	<i>HyperText Transfer Protocol</i> — protocole de transfert hypertexte
JSON	<i>JavaScript Object Notation</i> — format d'échange de données structurées
JWT	<i>JSON Web Token</i> — jeton d'authentification signé
LLM	<i>Large Language Model</i> — grand modèle de langage
LoRA	<i>Low-Rank Adaptation</i> — méthode d'ajustement fin paramétrique à faible rang
ML	<i>Machine Learning</i> — apprentissage automatique
MLOps	<i>Machine Learning Operations</i> — ensemble des pratiques d'industrialisation des systèmes d'apprentissage automatique
OCR	<i>Optical Character Recognition</i> — reconnaissance optique de caractères
OTLP	<i>OpenTelemetry Protocol</i> — protocole d'échange de données de télémétrie
P50 / P95	Percentiles 50 et 95 d'une distribution de latence
PII	<i>Personally Identifiable Information</i> — données à caractère personnel permettant d'identifier une personne
PromQL	<i>Prometheus Query Language</i> — langage d'interrogation des métriques
QLoRA	<i>Quantized LoRA</i> — variante quantifiée de LoRA
RAG	<i>Retrieval-Augmented Generation</i> — génération augmentée par récupération
RAGAS	<i>RAG Assessment</i> — cadre d'évaluation automatisée des systèmes RAG
RGAA	Référentiel général d'amélioration de l'accessibilité
RRF	<i>Reciprocal Rank Fusion</i> — fusion par rang réciproque
RTL	<i>Right-To-Left</i> — sens d'écriture de droite à gauche (arabe)
SHA-256	<i>Secure Hash Algorithm 256 bits</i> — fonction de hachage cryptographique

Sigle	Signification
SPA	<i>Single-Page Application</i> — application monopage
SRE	<i>Site Reliability Engineering</i> — ingénierie de la fiabilité des sites
SSE	<i>Server-Sent Events</i> — flux d'évènements émis par le serveur
TSDB	<i>Time Series Database</i> — base de données de séries temporelles
UML	<i>Unified Modeling Language</i> — langage de modélisation unifié
URL	<i>Uniform Resource Locator</i> — adresse de ressource
WCAG	<i>Web Content Accessibility Guidelines</i> — règles d'accessibilité des contenus web

Chapitre 1

Introduction générale

1.1 Contexte et problématique

L'accès au droit constitue, au Maroc comme ailleurs, un enjeu à la fois technique et social. Les textes de loi, la jurisprudence et les modèles contractuels forment un corpus volumineux, hétérogène et bilingue, dont l'exploitation manuelle est coûteuse en temps pour les professionnels du droit comme pour les citoyens. L'apparition des grands modèles de langage a fait naître l'espoir d'automatiser cette recherche documentaire ; elle a également révélé une difficulté majeure, qui est le point de départ de ce projet.

En effet, un grand modèle de langage interrogé directement sur une question juridique produit une réponse fluide et plausible, mais sans garantie d'exactitude : il peut citer un article de loi qui n'existe pas, ou attribuer à un texte réel un contenu qu'il ne comporte pas. Ce phénomène, désigné sous le terme d'*hallucination*, est structurellement incompatible avec le domaine juridique, où une citation erronée n'est pas une imprécision mais une faute.

La problématique de ce projet peut donc être formulée ainsi :

Comment concevoir et industrialiser un assistant juridique fondé sur l'intelligence artificielle générative, qui garantisse structurellement la provenance vérifiable de chacune de ses réponses, tout en respectant le cadre réglementaire applicable à la protection des données personnelles ?

Cette formulation contient deux exigences distinctes, qui structurent l'intégralité du travail présenté dans ce rapport.

1.1.1 Première exigence : la provenance vérifiable

Il ne suffit pas que le système produise une réponse juste : il doit pouvoir *démontrer* d'où provient cette réponse. Cette exigence exclut d'emblée l'usage d'un modèle de langage interrogé seul, et oriente le projet vers l'architecture RAG, détaillée au chapitre 2, dans laquelle le modèle ne génère qu'à partir de passages effectivement récupérés dans le corpus.

1.1.2 Seconde exigence : la conformité réglementaire

Tout corpus juridique réel contient des données à caractère personnel : noms de parties, adresses, identifiants. La loi marocaine 09-08 [12] impose un cadre strict à leur traitement. Cette contrainte n'est pas un supplément administratif : elle détermine des choix d'architecture concrets, notamment le *placement* de l'anonymisation dans le pipeline d'ingestion (§6.1) et la purge des données personnelles avant stockage dans la télémétrie (§6.7).

1.2 Objectif du projet

L'objectif fixé pour la version 0.2.0 n'est pas de livrer un produit fini, mais de livrer un **système RAG juridique de bout en bout, entièrement conteneurisé, observable et conforme**, validé par des pratiques d'ingénierie logicielle rigoureuses.

Le mot clé est *bout en bout*. Plutôt que de pousser un seul module très loin pendant que les autres restent à l'état d'ébauche, l'équipe a choisi de faire circuler une question juridique réelle depuis l'ingestion du corpus jusqu'à l'affichage d'une réponse citée dans l'interface, quitte à ce que chaque maillon de cette chaîne porte des limites explicitement documentées. Ce choix méthodologique, discuté au chapitre 4, a une conséquence directe sur la lecture de ce rapport : les limites y occupent autant de place que les réalisations, et sont présentées avec le même niveau de détail.

1.3 Organisation du travail

Le projet est découpé en huit modules (M1 à M8), chacun porté par un membre responsable au sein de l'équipe CloudMind, avec un relecteur désigné. Le dépôt suit un flux de travail GitHub structuré : branches de fonctionnalité, revues de code obligatoires via un fichier CODEOWNERS organisé en quatre équipes transverses, et un jeu de contrôles bloquants exécutés automatiquement sur chaque contribution.

Cette organisation, détaillée au chapitre 4, explique la cohérence du niveau de preuve apporté à chaque module : un travail n'est présenté comme « réalisé » dans ce rapport que s'il est couvert par des tests automatisés vérifiables, et non sur simple déclaration de son auteur.

1.4 Structure du rapport

Ce document est organisé en dix chapitres, articulés en quatre temps.

- **Les fondements** — le chapitre 2 expose l'état de l'art et les notions théoriques mobilisées (MLOps, RAG, recherche hybride, observabilité), et justifie les choix d'architecture au regard des alternatives existantes.
- **La conception** — le chapitre 3 formalise les besoins fonctionnels et non fonctionnels et présente la modélisation du système; le chapitre 4 décrit la méthodologie de gestion de projet et la planification effectivement suivie.
- **La réalisation** — le chapitre 5 présente l'architecture globale, le chapitre 6 détaille les huit modules selon une grille de lecture uniforme, et le chapitre 7 présente les résultats obtenus, captures d'écran à l'appui.
- **Le bilan** — le chapitre 8 rassemble les décisions transverses et la carte des dépendances, et le chapitre 9 dresse le bilan du projet et sa trajectoire.

Pour chacun des huit modules, le chapitre 6 distingue systématiquement trois niveaux d'information : ce qui est *réalisé et vérifié* (avec la preuve correspondante), ce qui reste une *limite assumée*

(avec sa justification), et les *dépendances* qui relient ce module au reste du système. Cette structure vise un seul objectif : qu'aucune affirmation de ce rapport ne soit plus forte que la preuve qui la soutient.

Chapitre 2

État de l’art et fondements théoriques

Ce chapitre expose les notions théoriques mobilisées par le projet. Il ne s’agit pas d’un exposé encyclopédique : chaque notion présentée ici est celle qui justifie un choix d’architecture concret, détaillé plus loin dans le rapport. Le lecteur familier de ces notions peut se reporter directement au chapitre 3.

2.1 Le MLOps : définition et cycle de vie

2.1.1 Origine du besoin

La mise en production d’un modèle d’apprentissage automatique diffère fondamentalement de celle d’un logiciel classique. Sculley et al. [10] ont montré dès 2015 que le code d’apprentissage proprement dit ne représente qu’une fraction minoritaire d’un système d’apprentissage réel : l’essentiel de la complexité réside dans la collecte des données, la configuration, la surveillance et l’infrastructure de service.

Cette observation a donné naissance au **MLOps**, ensemble de pratiques visant à appliquer aux systèmes d’apprentissage automatique la discipline d’ingénierie déjà éprouvée par le DevOps : automatisation, reproductibilité, supervision et itération contrôlée.

Notion — dette technique cachée

Un système d’apprentissage automatique peut être fonctionnellement correct tout en étant *ingérable* : données non versionnées, résultats non reproductibles, absence de supervision. Cette dette n’apparaît pas dans les métriques de performance du modèle, mais se manifeste au premier incident de production. Le MLOps vise précisément à la rendre visible et à la maîtriser.

2.1.2 Le cycle de vie MLOps

Le cycle de vie MLOps est classiquement représenté comme une boucle fermée, plutôt que comme une séquence linéaire. La figure 2.1 en donne une représentation, sur laquelle sont positionnés les huit modules du projet.

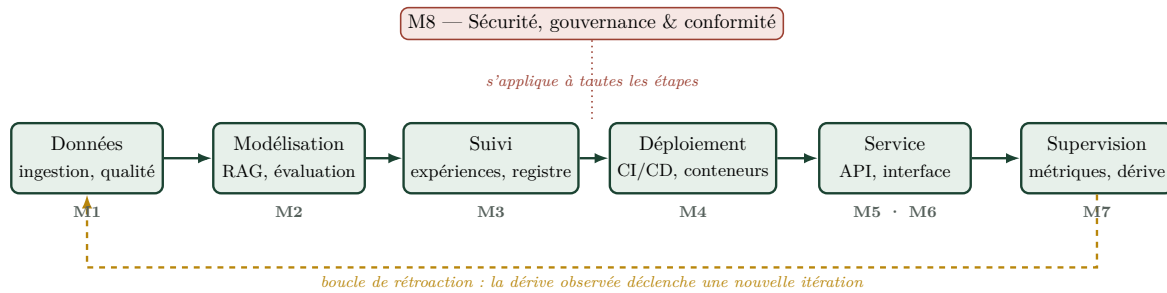


FIGURE 2.1 – Le cycle de vie MLOps, représenté comme une boucle fermée, et le positionnement des huit modules du projet. Le module M8 n’occupe pas une étape particulière : il s’applique transversalement à l’ensemble du cycle.

Trois caractéristiques de ce cycle méritent d’être soulignées, car elles expliquent des décisions prises dans ce projet :

- **C’est une boucle, pas une ligne.** La supervision n’est pas la dernière étape mais celle qui alimente la suivante itération. C’est la justification du module M7 (§6.7), dont le rôle n’est pas décoratif mais fonctionnel.
- **Les données sont un artefact versionné.** Au même titre que le code, elles doivent être reproductibles ; d’où l’usage de DVC dans M1.
- **La conformité est transverse.** Elle ne peut pas être ajoutée *a posteriori* à une étape donnée, ce qui explique pourquoi l’anonymisation est placée dans le pipeline d’ingestion et non après.

2.2 Les grands modèles de langage et leurs limites

2.2.1 Principe

Un grand modèle de langage repose sur l’architecture *Transformer* introduite par Vaswani et al. [2], dont le mécanisme central est l’*attention* : pour produire chaque nouveau mot, le modèle pondère l’ensemble des mots déjà présents dans son contexte. Entraîné sur d’immenses volumes de texte, il apprend à prédire le mot suivant le plus probable.

2.2.2 La limite structurelle : l’hallucination

Cette formulation contient la limite : le modèle optimise la *vraisemblance*, non la *véracité*. Un article de loi inventé mais formulé de façon plausible satisfait parfaitement l’objectif d’entraînement du modèle. De plus, les connaissances du modèle sont figées à la date de fin de son entraînement, ce qui est rédhibitoire pour un corpus juridique qui évolue.

Notion — hallucination

Production, par un modèle de langage, d'un énoncé syntaxiquement correct et sémantiquement plausible, mais factuellement faux ou sans fondement dans une source réelle. En droit, ce risque n'est pas atténuable par l'usage : une référence erronée invalide le raisonnement qui s'appuie sur elle.

Deux stratégies existent pour ancrer un modèle dans des connaissances vérifiées : l'*ajustement fin* (*fine-tuning*), qui modifie les poids du modèle, et la *génération augmentée par récupération* (RAG), qui lui fournit les sources au moment de la question. La section 2.4 compare ces deux approches et justifie le choix effectué.

2.3 La génération augmentée par récupération (RAG)

2.3.1 Principe général

Introduite par Lewis et al. [1], l'architecture RAG dissocie deux fonctions que le modèle de langage seul confondait : *retrouver* l'information et *formuler* la réponse. Le système recherche d'abord les passages pertinents dans un corpus contrôlé, puis demande au modèle de rédiger une réponse *uniquement à partir de ces passages*.

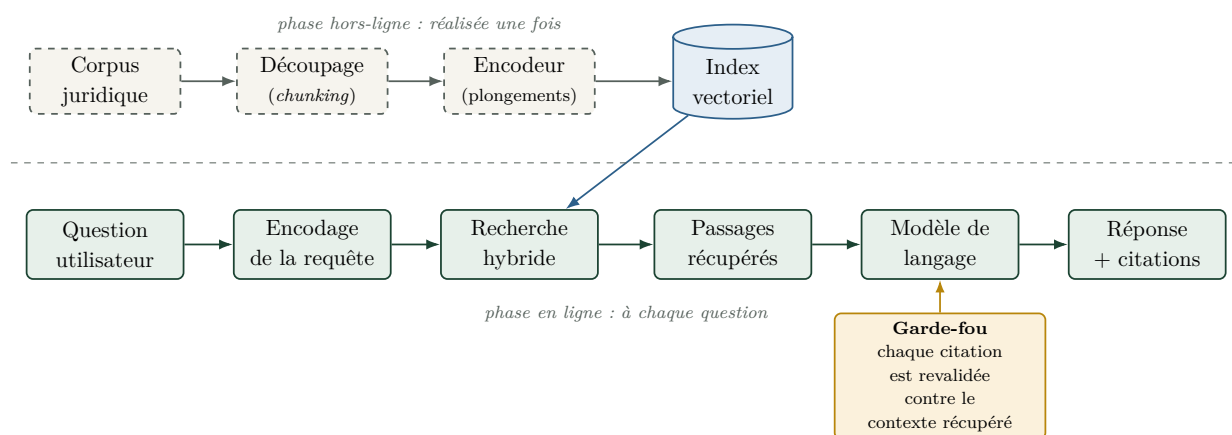


FIGURE 2.2 – Architecture d'un système RAG. La phase hors-ligne construit l'index une fois pour toutes ; la phase en ligne est exécutée à chaque question. Le garde-fou de validation des citations, ajouté par le module M2, ne fait pas partie de l'architecture RAG canonique.

2.3.2 Le découpage du corpus

Un texte juridique ne peut pas être indexé d'un seul tenant : la fenêtre de contexte d'un modèle est bornée, et un document entier diluerait l'information pertinente. Le corpus est donc segmenté en *fragments* (*chunks*) de taille contrôlée.

Ce découpage est un compromis : des fragments trop courts perdent le contexte nécessaire à l'interprétation d'un article de loi ; des fragments trop longs diluent le signal et saturent la fenêtre du modèle. Dans le cadre de ce projet, le découpage préserve les frontières structurales du texte juridique (article, alinéa), de façon à ce qu'un fragment reste une unité de sens citable.

2.3.3 Les plongements vectoriels

Un *plongement* (*embedding*) est la représentation d'un fragment de texte sous forme de vecteur numérique de dimension fixe, construite de telle sorte que deux textes de sens proche produisent des vecteurs proches dans l'espace.

La proximité est mesurée par la **similarité cosinus** entre deux vecteurs $\mathbf{u}$ et $\mathbf{v}$:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \sqrt{\sum_{i=1}^n v_i^2}} \quad (2.1)$$

Cette mesure varie entre -1 et 1 , la valeur 1 correspondant à deux vecteurs colinéaires, c'est-à-dire à deux textes jugés sémantiquement équivalents par le modèle d'encodage.

Le projet retient le modèle multilingue BGE-M3 [6], dont la particularité est de projeter le français et l'arabe dans un *même* espace vectoriel — propriété indispensable pour un corpus juridique marocain bilingue, puisqu'elle permet de retrouver un passage en arabe à partir d'une question posée en français.

2.3.4 Recherche lexicale : BM25

La recherche lexicale repose sur la correspondance de termes. La fonction BM25 [3] attribue à un document D , pour une requête Q composée des termes $q_1, \dots, q_n$, le score :

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (2.2)$$

où $f(q_i, D)$ est la fréquence du terme q_i dans D , $|D|$ la longueur du document, avgdl la longueur moyenne des documents du corpus, et k_1 et b des paramètres de calibration. Le facteur IDF (*inverse document frequency*) pondère les termes en fonction de leur rareté : un terme fréquent dans tout le corpus est peu discriminant.

2.3.5 Recherche dense

La recherche dense [5] procède différemment : la requête est encodée en vecteur, puis comparée aux vecteurs de l'index par la similarité (2.1). Elle retrouve des passages pertinents même lorsque le vocabulaire diffère — par exemple une question sur le « congé » et un texte mentionnant le « préavis ».

2.3.6 Pourquoi combiner les deux : la fusion RRF

Les deux méthodes ont des défauts complémentaires. La recherche lexicale échoue sur les reformulations ; la recherche dense peut manquer une référence parfaitement littérale, comme un numéro d'article précis — ce qui serait rédhibitoire en droit.

Le projet retient donc une **recherche hybride**, dont les deux listes de résultats sont fusionnées par la méthode RRF (*Reciprocal Rank Fusion*) [4]. Le score d'un document d est :

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rang}_r(d)} \quad (2.3)$$

où R est l'ensemble des méthodes de recherche, $\text{rang}_r(d)$ le rang du document d selon la méthode r , et k une constante d'amortissement (typiquement $k = 60$).

L'intérêt de cette formule est qu'elle n'utilise que les *rangs*, jamais les scores bruts. C'est un avantage décisif : les scores BM25 et les similarités cosinus vivent sur des échelles incomparables, et toute tentative de les normaliser introduirait un paramètre arbitraire. La fusion par rangs évite entièrement cette difficulté.

2.4 RAG contre ajustement fin : justification d'un choix

L'ajustement fin paramétrique, dont LoRA [7] et sa variante quantifiée QLoRA [8] sont les méthodes les plus répandues, consiste à modifier un sous-ensemble des poids du modèle pour l'adapter à un domaine. Le tableau 2.1 compare les deux approches sur les critères qui importent pour ce projet.

Critère	RAG	Ajustement fin (LoRA/QLoRA)
Provenance de la réponse	Traçable : chaque passage est identifié	Non traçable : la connaissance est diluée dans les poids
Mise à jour du corpus	Ré-indexation, quelques minutes	Nouvel entraînement complet
Données nécessaires	Le corpus lui-même	Un jeu question-réponse annoté par des experts
Coût de calcul	Faible (inférence seule)	Élevé (entraînement sur processeur graphique)
Risque d'hallucination	Fortement réduit par la contrainte de contexte	Réduit sur le style, non sur les faits
Vérifiabilité par un juriste	Directe : les sources sont affichées	Impossible sans audit du modèle

TABLE 2.1 – Comparaison entre la génération augmentée par récupération et l'ajustement fin paramétrique, selon les critères pertinents pour un assistant juridique.

Décision de gouvernance

Conséquence pour ce projet. Le critère décisif est la *traçabilité* : une réponse juridique doit pouvoir être vérifiée par un professionnel du droit, ce que seul le RAG permet. À cela s'ajoute une contrainte matérielle : l'ajustement fin exige un jeu de données annoté par des experts juridiques, qui n'existe pas pour le corpus marocain visé. C'est cette double raison — une raison de principe et une raison de fait — qui a conduit au retrait formel du fine-tuning du périmètre de la version 0.2.0 (§6.2).

2.5 Les bases de données vectorielles

Comparer une requête à l'intégralité des vecteurs d'un index par un calcul exhaustif devient impraticable dès que le corpus grandit. Les bases vectorielles résolvent ce problème par des al-

algorithmes de *recherche approximative des plus proches voisins*, qui acceptent une marge d'erreur infime en échange d'un gain de temps de plusieurs ordres de grandeur.

Le projet retient **Qdrant**, pour trois raisons : le filtrage par métadonnées permet de restreindre une recherche à une juridiction ou à un type de texte ; l'auto-hébergement garantit que le corpus juridique ne quitte jamais l'infrastructure maîtrisée, ce qui est une exigence de conformité ; et la suppression par identifiant de document permet d'honorer un droit à l'effacement sans reconstruire l'index entier.

2.6 L'observabilité : les trois piliers

L'observabilité, issue des pratiques SRE formalisées par Beyer et al. [11], se distingue de la simple surveillance. Surveiller, c'est vérifier des conditions connues à l'avance ; observer, c'est être capable de *diagnostiquer une panne que l'on n'avait pas anticipée*. Elle repose sur trois signaux complémentaires, présentés en figure 2.3.

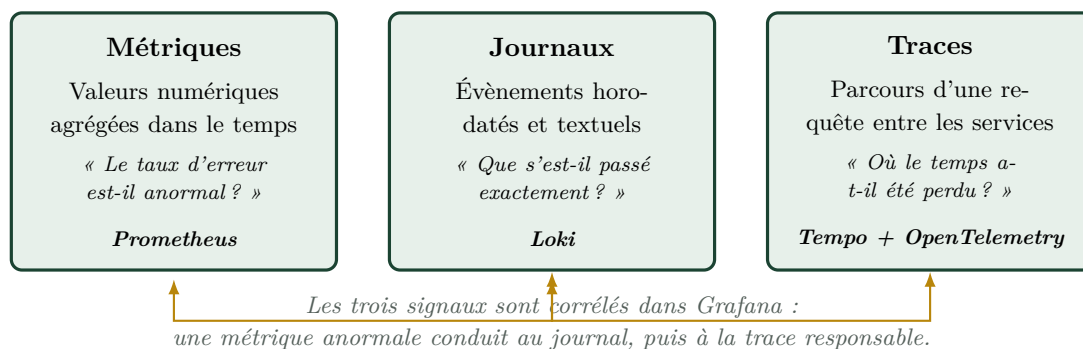


FIGURE 2.3 – Les trois piliers de l'observabilité et les technologies retenues par le module M7. Leur complémentarité est le point essentiel : aucun des trois ne suffit seul à diagnostiquer un incident.

Notion — échantillonnage des traces

Conserver la trace de chaque requête est coûteux. Deux stratégies existent : le *head sampling* décide de conserver ou non la trace *avant* l'exécution de la requête (simple, mais aveugle — il peut écarter précisément la requête lente qui nous intéresse) ; le *tail sampling* décide *après*, en fonction des caractéristiques observées (plus coûteux, mais il permet de ne retenir que les traces lentes ou en erreur). Ce choix est discuté en §6.7.

2.7 Intégration et livraison continues

L'intégration continue consiste à valider automatiquement chaque contribution au code par une batterie de contrôles, avant toute fusion. La livraison continue prolonge ce principe jusqu'à la production d'un artefact déployable.

L'apport essentiel de cette pratique, dans un projet à huit contributeurs, n'est pas le gain de temps mais la *suppression de l'argument d'autorité* : une contribution n'est pas acceptée parce que son auteur affirme qu'elle fonctionne, mais parce qu'un ensemble de contrôles reproductibles l'a vérifié.

C'est directement ce mécanisme qui permet, dans ce rapport, de distinguer une réalisation prouvée d'une déclaration d'intention.

2.8 La dérive des données et des modèles

Un modèle déployé se dégrade dans le temps, sans qu'aucune ligne de code ne change. On distingue :

- la **dérive des données** (*data drift*) : la distribution des questions posées change — par exemple une évolution du vocabulaire juridique ou l'arrivée d'un nouveau domaine de questions ;
- la **dérive conceptuelle** (*concept drift*) : la relation entre question et bonne réponse change — typiquement lorsqu'une réforme législative rend caduque une réponse auparavant correcte.

Cette seconde forme est particulièrement critique en droit, où une modification d'un texte invalide instantanément toutes les réponses qui s'y référaient. Sa détection suppose toutefois de disposer d'un volume de trafic réel : sur un échantillon trop faible, les tests statistiques ne distinguent pas une dérive d'une fluctuation aléatoire. Cette contrainte explique la limite documentée en §6.7.

2.9 Étude de l'existant

Plusieurs catégories de solutions existent pour l'assistance juridique automatisée. Le tableau 2.2 les situe par rapport au positionnement retenu.

Catégorie	Limites au regard de notre problématique	Positionnement du projet
Moteurs de recherche juridiques classiques	Retrouvent des documents, mais ne synthétisent pas de réponse ; exigent la connaissance préalable du vocabulaire juridique exact	Conserve la recherche lexicale (BM25) mais y ajoute la synthèse et la reformulation
Assistants généralistes fondés sur un LLM seul	Aucune garantie de provenance ; connaissances figées ; corpus marocain marginalement représenté	Ancre systématiquement la réponse dans un corpus contrôlé et cite ses sources
Solutions commerciales de <i>legal tech</i>	Corpus centrés sur les droits français ou anglo-saxon ; hébergement externe incompatible avec les contraintes de souveraineté des données	Corpus marocain bilingue ; infrastructure entièrement auto-hébergée
Prototypes RAG académiques	Démontrent la faisabilité mais restent non industrialisés : ni supervision, ni conformité, ni reproductibilité	Industrialisation complète : c'est précisément l'objet des modules transverses M3, M4, M7 et M8

TABLE 2.2 – Positionnement du projet par rapport aux catégories de solutions existantes.

Ce positionnement délimite la contribution propre du projet : elle ne réside pas dans l'invention d'un algorithme nouveau — les briques mobilisées sont toutes documentées dans la littérature — mais dans leur *assemblage industrialisé* sous une double contrainte de provenance vérifiable et de conformité réglementaire, appliquée à un corpus juridique marocain bilingue.

Chapitre 3

Analyse et conception

Ce chapitre formalise ce que le système doit faire, puis comment il est structuré pour y parvenir. Il précède volontairement la description de l'architecture réalisée (chapitre 5) : les besoins exposés ici sont ceux qui ont guidé les choix, non leur justification rétrospective.

La modélisation s'appuie sur le langage UML, selon cinq vues complémentaires : une vue *fonctionnelle* (cas d'utilisation), une vue *dynamique* (séquence et activité), une vue *statique* (classes) et une vue *physique* (déploiement).

3.1 Analyse des besoins fonctionnels

Les besoins fonctionnels décrivent les services attendus du système. Ils ont été établis à partir de deux profils d'utilisateur distincts : le *juriste*, qui consulte et vérifie, et l'*administrateur technique*, qui supervise et maintient.

Réf.	Besoin fonctionnel	Module
BF-01	Poser une question juridique en langage naturel, en français ou en arabe	M5 · M6
BF-02	Obtenir une réponse synthétique accompagnée de ses citations sources	M2
BF-03	Vérifier chaque citation en consultant le passage d'origine	M2 · M6
BF-04	Obtenir un refus explicite lorsque la question sort du périmètre couvert	M2
BF-05	Téléverser un document contractuel en vue de son analyse	M5 · M6
BF-06	Consulter l'historique de ses propres consultations	M6
BF-07	Basculer l'interface entre le français et l'arabe, sens d'écriture compris	M6
BF-08	Ingérer un nouveau corpus de façon reproductible et tracée	M1
BF-09	Comparer les performances de plusieurs configurations du système	M3
BF-10	Superviser la santé du système et être alerté en cas d'anomalie	M7
BF-11	Auditer les actions effectuées, sans exposer de données personnelles	M8

TABLE 3.1 – Besoins fonctionnels du système et modules responsables de leur réalisation.

Le besoin BF-04 mérite un commentaire, car il est contre-intuitif : la capacité à *refuser* de répondre est ici un besoin fonctionnel à part entière, et non un cas d'erreur. Un assistant juridique qui répond à tout, y compris hors de son corpus, est plus dangereux qu'un assistant qui reconnaît ses limites.

3.2 Analyse des besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues du système. Ils sont ici particulièrement structurants, puisque trois d'entre eux (BNF-01, BNF-04 et BNF-05) découlent directement de la problématique.

Réf.	Catégorie	Exigence
BNF-01	Fiabilité	Toute citation produite doit correspondre mot pour mot à un passage effectivement récupéré ; à défaut, elle est rejetée
BNF-02	Performance	Latence de récupération visée sous les 120 ms au percentile 50
BNF-03	Disponibilité	Le système doit pouvoir être redéployé intégralement à partir du dépôt, sans intervention manuelle
BNF-04	Confidentialité	Aucune donnée personnelle ne doit être écrite dans les journaux, les traces ou les URL
BNF-05	Conformité	Traitement conforme à la loi 09-08 ; anonymisation appliquée avant tout stockage
BNF-06	Sécurité	Authentification obligatoire sur toutes les routes sensibles ; aucun secret dans le dépôt
BNF-07	Accessibilité	Interface conforme aux règles WCAG 2.1 niveau AA ; prise en charge du sens d'écriture de droite à gauche
BNF-08	Maintenabilité	Toute contribution validée par des contrôles automatisés bloquants avant fusion
BNF-09	Observabilité	Tout incident doit être diagnosticable <i>a posteriori</i> à partir des signaux collectés
BNF-10	Portabilité	L'ensemble des services doit être conteneurisé et démarrable par une commande unique

TABLE 3.2 – Besoins non fonctionnels du système.

3.3 Vue fonctionnelle : diagramme de cas d'utilisation

La figure 3.1 formalise les interactions entre les acteurs et le système. Deux acteurs humains sont identifiés, ainsi qu'un acteur secondaire non humain — le fournisseur du modèle de langage — dont l'explicitation est volontaire : elle rappelle qu'il s'agit d'un service externe, dont la défaillance ou l'indisponibilité doit être traitée comme un cas nominal.

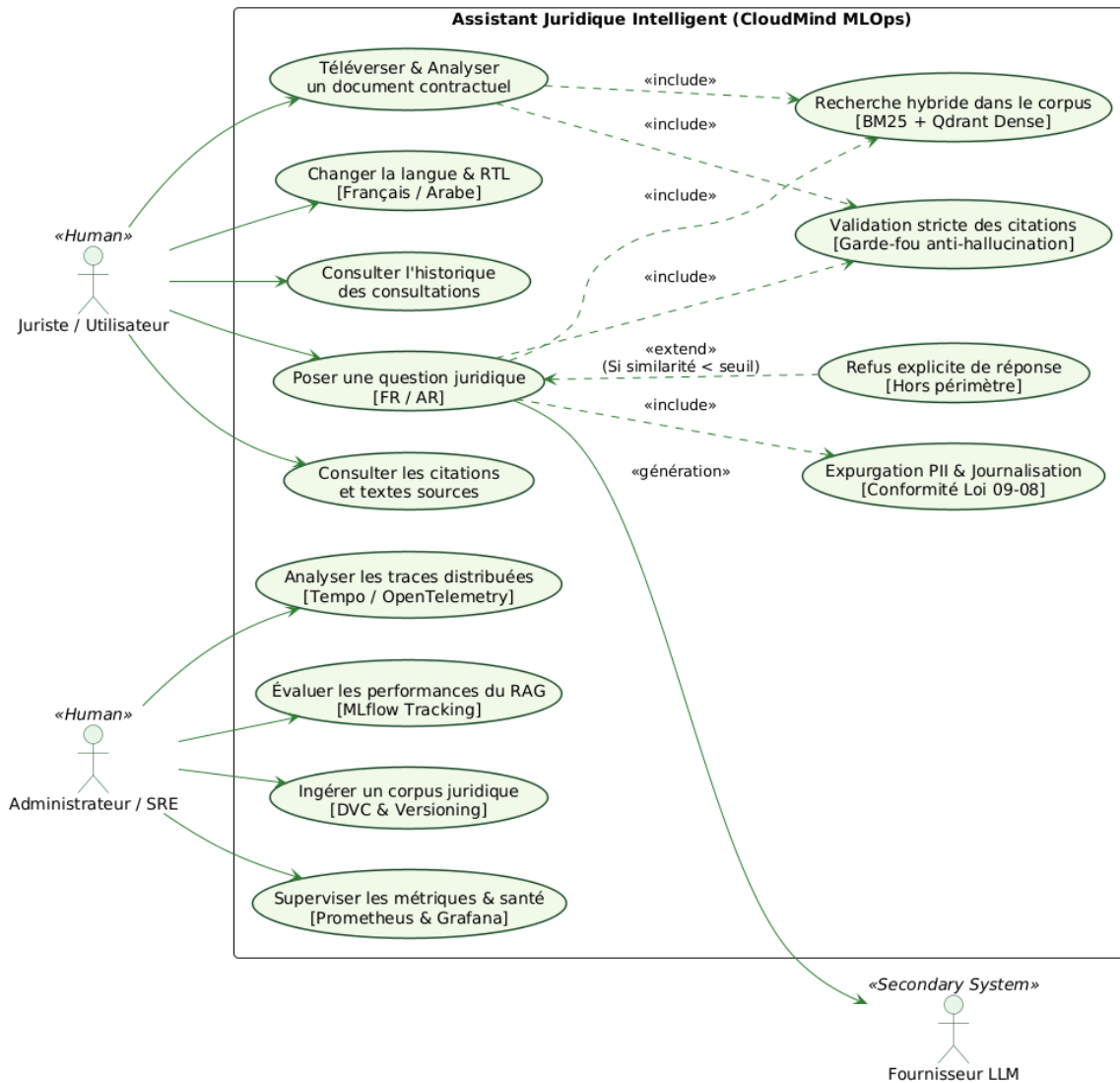


FIGURE 3.1 – Diagramme de cas d'utilisation du système. Les cas reliés par *«include»* sont systématiquement exécutés et non optionnels ; le refus explicite est modélisé par une relation *«extend»*, déclenchée lorsque le score de similarité passe sous le seuil minimal.

Trois éléments de ce diagramme méritent d'être soulignés, car ils traduisent directement les exigences énoncées plus haut :

- **La validation des citations est incluse, non optionnelle.** Le garde-fou anti-hallucination est rattaché au cas « poser une question juridique » par une relation d'inclusion : il n'existe aucun chemin d'exécution qui permette de l'ignorer. C'est la traduction du besoin BNF-01.
- **Le refus est un comportement modélisé.** Le cas « refus explicite de réponse » n'est pas un cas d'erreur mais une extension nominale, ce qui matérialise le besoin BF-04.
- **L'expurgation est un cas d'utilisation à part entière.** La conformité à la loi 09-08 apparaît dans la vue fonctionnelle et non seulement dans l'implémentation, conformément

au principe de conformité transverse exposé au chapitre 2.

3.4 Vue dynamique : diagramme de séquence

La figure 3.2 détaille le parcours complet d’une question juridique à travers les composants du système, en distinguant les deux issues possibles : le refus pour cause de hors-périmètre, et la génération d’une réponse validée.

Quatre étapes de cette séquence sont propres à ce projet et absentes d’une chaîne RAG standard :

- **Étape 2** — la question est transmise dans le corps de la requête HTTP et jamais dans l’URL, car une URL peut être journalisée ou mise en cache sans que son contenu soit perçu comme sensible (BNF-04).
- **Étape 3** — le segment de trace est ouvert *déjà expurgé* de toute donnée personnelle, et non nettoyé après coup.
- **Étapes 13 à 15** — chaque citation produite par le modèle est confrontée mot pour mot au contexte récupéré ; toute citation introuvable est rejetée ou supprimée avant l’envoi de la réponse.
- **Étape 18** — les métriques transmises à l’observabilité portent sur la latence, les jetons et le statut de cache, mais jamais sur le texte de la question.

3.5 Vue dynamique : diagramme d’activité

Là où le diagramme de séquence montre les échanges *entre* composants, le diagramme d’activité de la figure 3.3 détaille la logique *interne* du traitement, en particulier les points de décision et la boucle de validation.

Ce diagramme rend visibles deux mécanismes que les autres vues ne montrent pas.

Le parallélisme de la recherche hybride. La recherche dense et la recherche lexicale ne s’exécutent pas l’une après l’autre mais simultanément, puis leurs listes sont fusionnées par RRF selon l’équation (2.3). Ce choix d’implémentation est ce qui permet de viser l’objectif de latence fixé par le besoin BNF-02 : une exécution séquentielle additionnerait les deux temps de recherche.

La boucle de validation comme source de métrique. Lorsqu’une citation est rejetée, le système ne se contente pas de la supprimer : il enregistre l’incident dans les métriques. Une hallucination détectée devient donc un signal observable, exploitable par le module M7 — ce qui transforme un défaut ponctuel en indicateur de qualité suivi dans le temps.

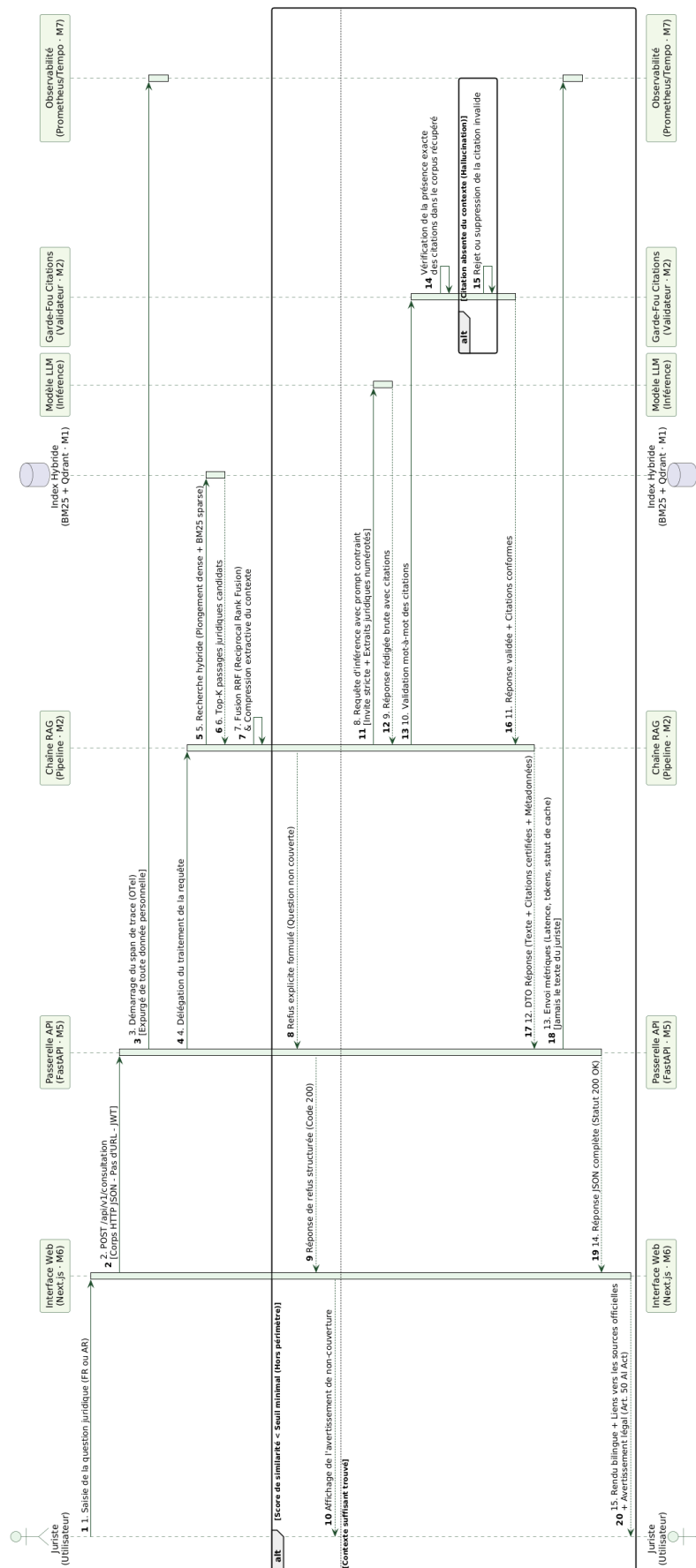


FIGURE 3.2 – Parcours d’une question juridique à travers les huit participants du système. Le bloc *alt* supérieur distingue le refus hors-périmètre de la génération nominale; le bloc *alt* imbriqué, à droite, traite le cas d’une citation absente du contexte — c’est-à-dire d’une hallucination détectée.

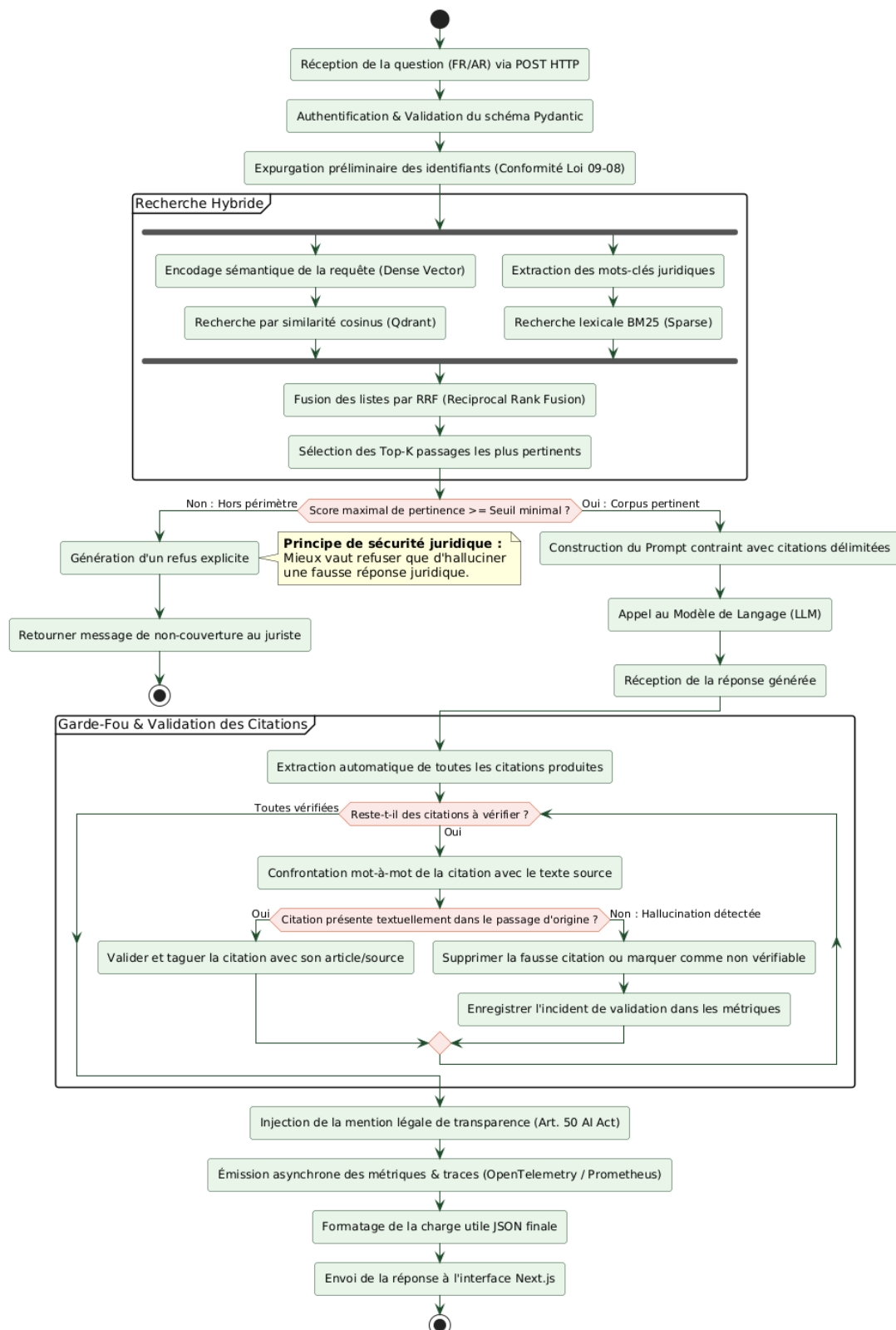


FIGURE 3.3 – Logique interne du traitement d’une question. Le bloc « Recherche hybride » exécute en parallèle la recherche dense et la recherche lexicale avant de fusionner les résultats; le bloc « Garde-fou » itère sur chaque citation produite et enregistre tout rejet comme incident mesurable.

Décision de gouvernance

Le diagramme porte une note explicite : « *mieux vaut refuser que d'halluciner une fausse réponse juridique* ». Cette formulation n'est pas un commentaire décoratif : elle justifie que la branche de refus soit un chemin nominal du diagramme, au même titre que la branche de génération.

3.6 Vue statique : diagramme de classes

La figure 3.4 présente le modèle de données de la chaîne RAG, c'est-à-dire les entités manipulées et leurs relations.

Trois choix de modélisation traduisent les exigences du projet dans la structure même des données :

- **L'anonymisation est une méthode de la requête.** La classe `LegalQuery` porte une opération `anonymiser_pii()` : le traitement des données personnelles n'est pas délégué à un service annexe, il est attaché à l'objet qui les transporte.
- **La vérification est un attribut persistant.** La classe `CitationSource` porte un champ `est_verifiee` et un `score_similarite` : le statut de vérification d'une citation accompagne la donnée et n'est pas recalculé à l'affichage.
- **Le hors-périmètre est une donnée, pas une exception.** La classe `LegalResponse` porte un champ `est_hors_perimetre` : un refus est une réponse structurée et mesurable, cohérent avec le besoin BF-04.

Enfin, la présence du champ `tolerance_levenshtein` dans `CitationValidator` mérite un mot : la validation des citations n'exige pas une égalité binaire mais une correspondance à une tolérance près, afin d'absorber les différences typographiques mineures sans pour autant accepter une reformulation — qui, elle, serait une hallucination.

3.7 Vue physique : diagramme de déploiement

La figure 3.5 présente la répartition effective des composants en conteneurs, leurs réseaux d'appartenance et les ports d'échange.

3.7.1 La règle de sécurité fondamentale

Ce diagramme matérialise le principe de surface d'exposition minimale :

- Les conteneurs de la pile d'observabilité — Prometheus, Loki, Tempo, Alertmanager, le collecteur OpenTelemetry et l'agent Promtail — sont liés à 127.0.0.1. Ce choix n'est pas arbitraire : aucun de ces services ne dispose d'authentification native, et Loki donne accès au journal d'audit.
- **Grafana** est le seul service d'observabilité exposé, précisément parce qu'il porte une au-

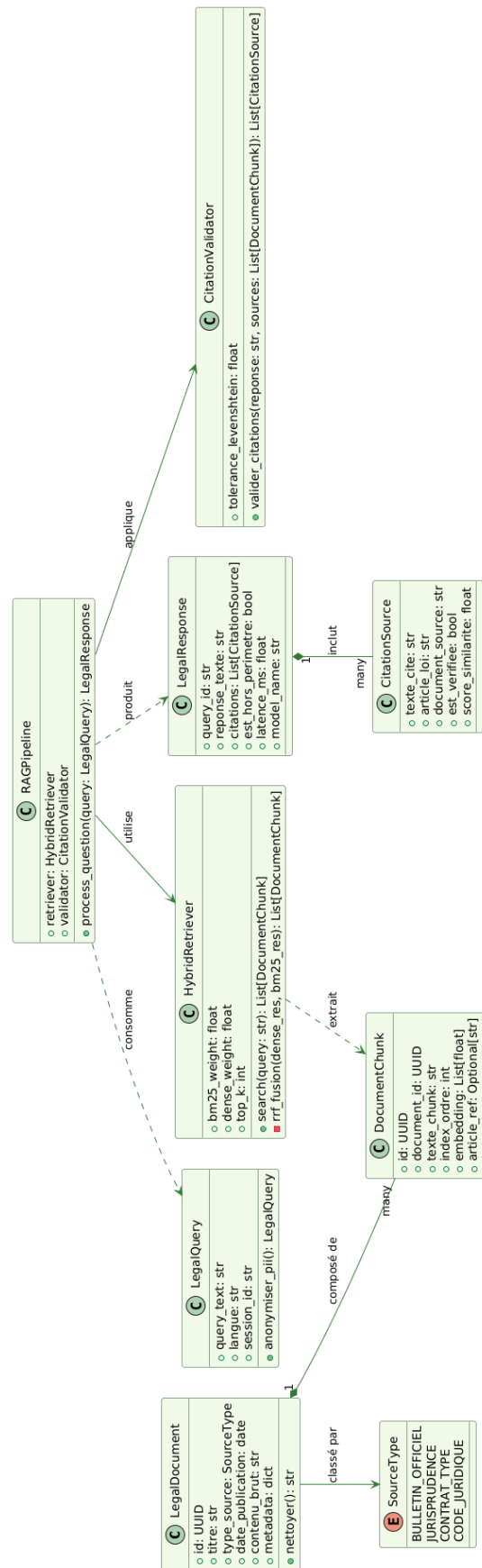


FIGURE 3.4 – Modèle statique de la chaîne RAG. La classe RAGPipeline orchestre quatre collaborations : elle consomme une LegalQuery, utilise un HybridRetriever, produit une LegalResponse et applique un CitationValidator.

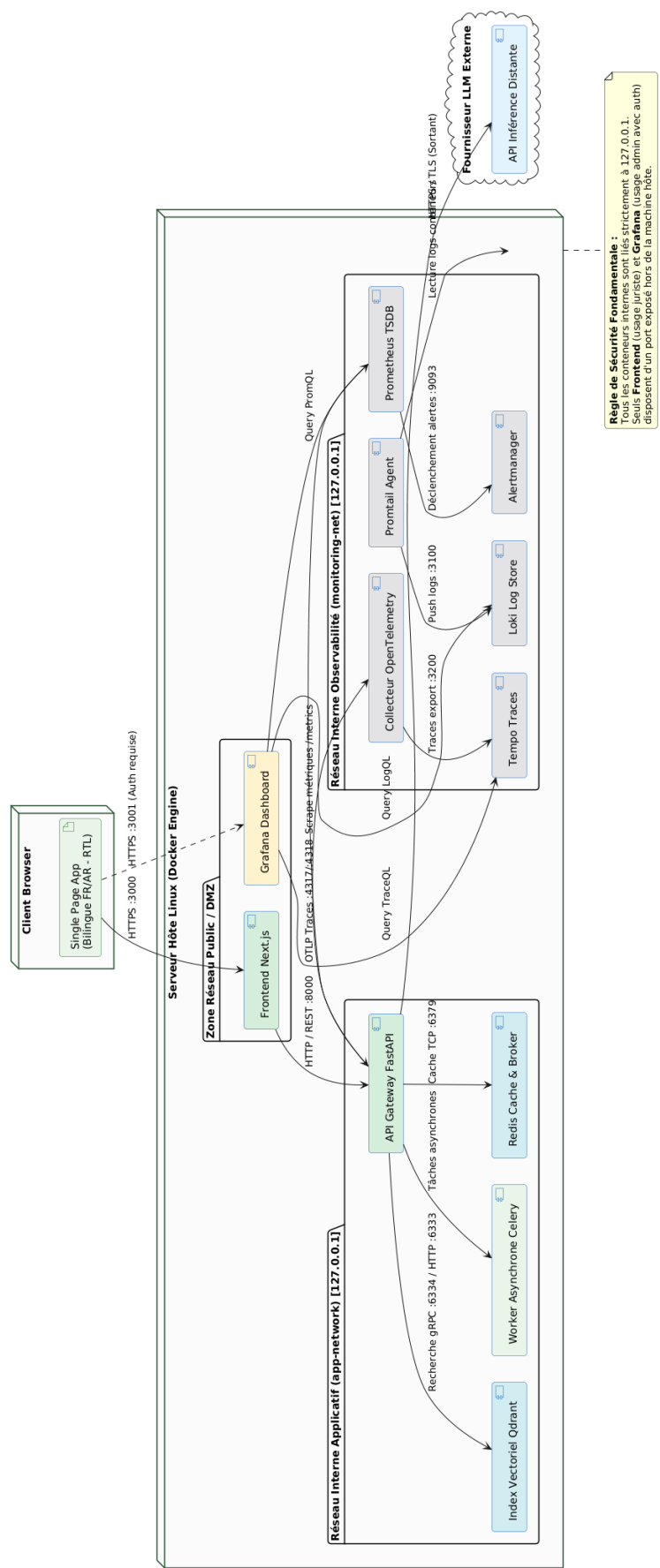


FIGURE 3.5 – Architecture physique déployée. Deux réseaux internes distincts isolent la chaîne applicative de la pile d’observabilité ; seuls le frontend et Grafana disposent d’un port exposé hors de la machine hôte, tous les autres conteneurs étant strictement liés à l’adresse de boucle locale.

thentification. Il sert de point d'entrée unique vers les trois signaux (§2.6), qu'il interroge par PromQL, LogQL et TraceQL.

- Le **frontend** est exposé au titre de sa fonction : c'est l'interface destinée au juriste. Il ne donne accès à aucune donnée d'exploitation.

3.7.2 La séparation des réseaux

Le déploiement distingue deux réseaux internes, **app-network** et **monitoring-net**. Cette séparation garantit qu'un composant d'observabilité ne peut pas atteindre directement l'index vectoriel ou le cache applicatif : la télémétrie circule dans un sens unique, depuis l'application vers le collecteur, et jamais l'inverse.

3.8 Synthèse de la conception

Trois principes de conception traversent les cinq vues présentées dans ce chapitre et se retrouveront à l'identique dans la réalisation :

1. **La vérifiabilité est structurelle, non optionnelle.** Elle apparaît comme cas inclus dans la vue fonctionnelle (figure 3.1), comme bloc de validation dans la vue dynamique (figures 3.2 et 3.3), et comme attribut persistant dans la vue statique (figure 3.4).
2. **La donnée personnelle ne circule pas.** Transmission par le corps de la requête, expurgation à l'ouverture du segment de trace, méthode d'anonymisation portée par la requête elle-même : le mécanisme est placé au plus tôt dans chacune des vues.
3. **La surface d'exposition est minimale.** Sur les onze conteneurs déployés, deux seulement sont joignables de l'extérieur, et chacun pour une raison explicite (figure 3.5).

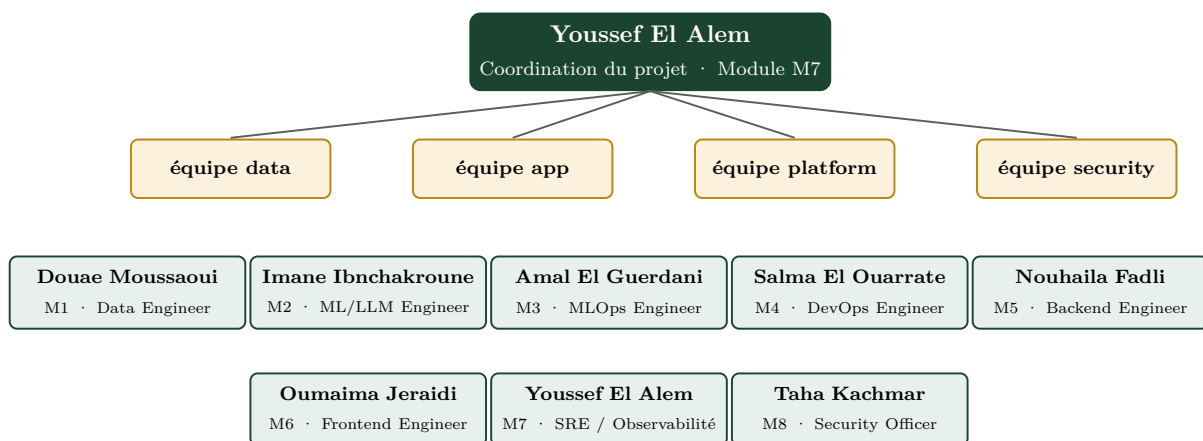
Chapitre 4

Méthodologie et gestion de projet

Un projet mené à huit contributeurs sur sept semaines ne se pilote pas comme un travail individuel. Ce chapitre décrit l'organisation adoptée, la méthodologie de développement retenue et la planification effectivement suivie. Il éclaire une question que le jury pourrait légitimement poser : comment garantir la cohérence technique d'un système dont les huit composants ont été développés en parallèle par des personnes différentes ?

4.1 Organisation de l'équipe

L'équipe est organisée autour d'un principe simple : à chaque module correspond un *responsable* unique, qui en répond devant le groupe, et un ensemble de relecteurs désignés par appartenance à une équipe transverse. La figure 4.1 présente cette organisation.



Les quatre équipes transverses (data, app, platform, security) déterminent, via le fichier CODEOWNERS, quelles personnes doivent approuver une contribution selon les fichiers qu'elle modifie. Une contribution touchant plusieurs domaines requiert l'accord de plusieurs équipes.

FIGURE 4.1 – Organisation de l'équipe. La responsabilité d'un module est individuelle ; la validation d'une contribution est collective.

4.2 Méthodologie de développement

4.2.1 Le choix du GitHub Flow

Parmi les méthodologies de gestion de branches, le projet retient le **GitHub Flow** plutôt que le *GitFlow* classique. Ce choix mérite d'être justifié : GitFlow, avec ses branches de développement, de publication et de correction distinctes, est conçu pour des logiciels livrés par versions espacées. Sur un cycle de sept semaines avec intégration quotidienne, il aurait introduit une lourdeur sans contrepartie.

Le flux retenu est représenté en figure 4.2.

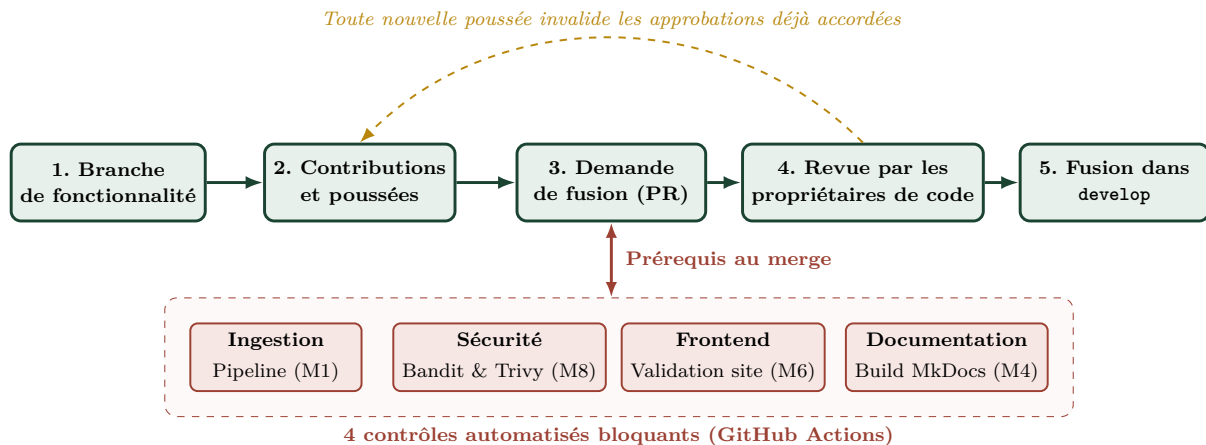


FIGURE 4.2 – Flux de contribution appliqué au projet. Les quatre contrôles automatisés sont bloquants : aucune fusion dans `develop` n’est techniquement possible tant qu’ils ne sont pas tous au vert. Toute modification ultérieure invalide les approbations de code déjà accordées.

4.2.2 Les garde-fous de gouvernance

Trois mécanismes complètent ce flux et méritent d’être détaillés, car ils expliquent la fiabilité du niveau de preuve revendiqué dans ce rapport :

- **Les propriétaires de code (CODEOWNERS)** : chaque répertoire du dépôt est rattaché à une ou plusieurs équipes. Une contribution modifiant des fichiers relevant de deux domaines exige l’approbation des deux équipes concernées — ce qui empêche structurellement qu’un module soit modifié à l’insu de son responsable.
- **L’invalidation des revues obsolètes** : toute nouvelle poussée sur une branche annule les approbations déjà accordées. Ce réglage évite qu’une contribution approuvée soit modifiée avant fusion.
- **Les quatre contrôles bloquants** : tests du pipeline d’ingestion, analyse de sécurité, validation du site statique et construction de la documentation. Leur caractère bloquant est le fondement de la distinction, dans ce rapport, entre « réalisé et vérifié » et « limite assumée ».

4.3 Planification du projet

4.3.1 Découpage temporel

Le projet s’est déroulé du **3 août** au **18 septembre 2026**, soit sept semaines de travail effectif, jalonnées par deux publications de version. La figure 4.3 présente le diagramme de Gantt correspondant.

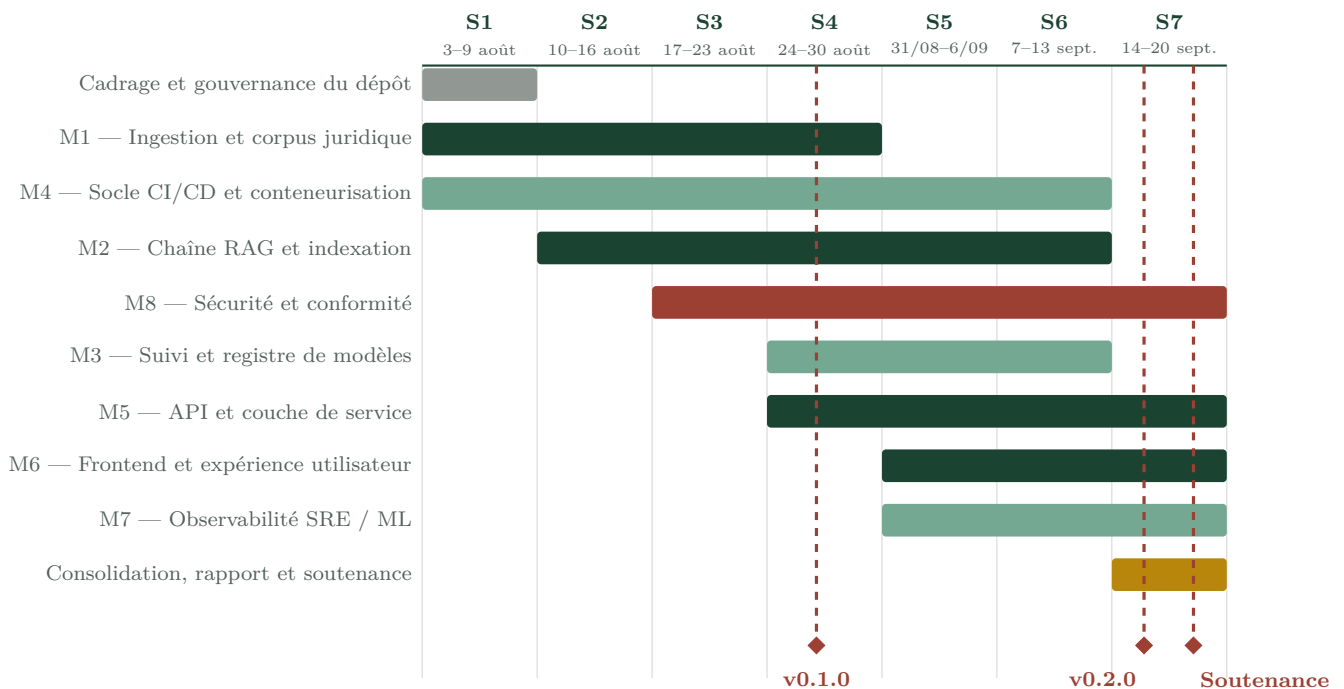


FIGURE 4.3 – Diagramme de Gantt. Les barres vertes correspondent aux modules du chemin critique, les barres claires aux modules transverses d’outillage, la barre rouge au module de conformité — actif dès la troisième semaine, avant même que le système ne soit fonctionnel.

4.3.2 Lecture du planning

Trois caractéristiques de cette planification méritent commentaire.

Le socle d’abord. Le module M4 (CI/CD) démarre dès la première semaine, avant tout développement fonctionnel. Ce choix, contre-intuitif au premier abord, découle du besoin BNF-08 : mettre en place les contrôles automatisés *après* avoir écrit le code aurait signifié valider rétrospectivement un travail déjà fait, ce qui vide le mécanisme de sa fonction.

La conformité en amont. Le module M8 démarre en troisième semaine, alors que le système n’est pas encore fonctionnel. C’est une application directe du principe exposé au chapitre 2 : la conformité étant transverse au cycle de vie, elle ne peut pas être ajoutée à la fin.

Le décalage des modules avals. Les modules M6 et M7 ne démarrent qu’en cinquième semaine, car ils dépendent respectivement du contrat d’interface de M5 et de l’existence d’un service à observer. Ce décalage, visible sur le diagramme, est la traduction temporelle de la carte de dépendances présentée au chapitre 8.

4.3.3 Rythme effectif des contributions

Le tableau 4.1 donne le volume de contributions enregistrées semaine par semaine dans le dépôt. Il complète le diagramme de Gantt en montrant la charge réelle, et non seulement la planification.

Semaine	Période	Contributions	Fait marquant
S1	3 — 9 août	17	Structuration du dépôt, gouvernance GitHub
S2	10 — 16 août	0	Interruption (période d'examens)
S3	17 — 23 août	2	Reprise progressive
S4	24 — 30 août	26	Publication de la version 0.1.0
S5	31 août — 6 sept.	31	Développement des modules applicatifs
S6	7 — 13 sept.	42	Intégration, pic d'activité
S7	14 — 20 sept.	17	Version 0.2.0 et consolidation
Total		135	

TABLE 4.1 – Répartition effective des contributions au dépôt. Le creux de la deuxième semaine correspond à une période d'examens ; le pic de la sixième semaine correspond à la phase d'intégration entre modules, qui est structurellement la plus coûteuse.

4.4 Enseignements méthodologiques

Deux enseignements se dégagent de cette organisation, qu'il est honnête de formuler explicitement.

Le premier est positif : le couplage entre responsabilité individuelle d'un module et validation collective des contributions a effectivement empêché les régressions silencieuses. Aucune contribution n'a été fusionnée sans revue pendant les sept semaines.

Le second est plus nuancé : la phase d'intégration (semaine 6) a concentré l'essentiel de la charge, révélant que les contrats d'interface entre modules avaient été définis trop tardivement. Un projet reconduit gagnerait à figer ces contrats dès la deuxième semaine, avant le développement des implémentations qu'ils relient.

Architecture globale

5.1 Vue d'ensemble

L'architecture du système repose sur un **chemin critique** en quatre étapes, entouré de quatre modules transverses qui outillent, sécurisent et rendent observable ce chemin. La figure 5.1 synthétise cette organisation.

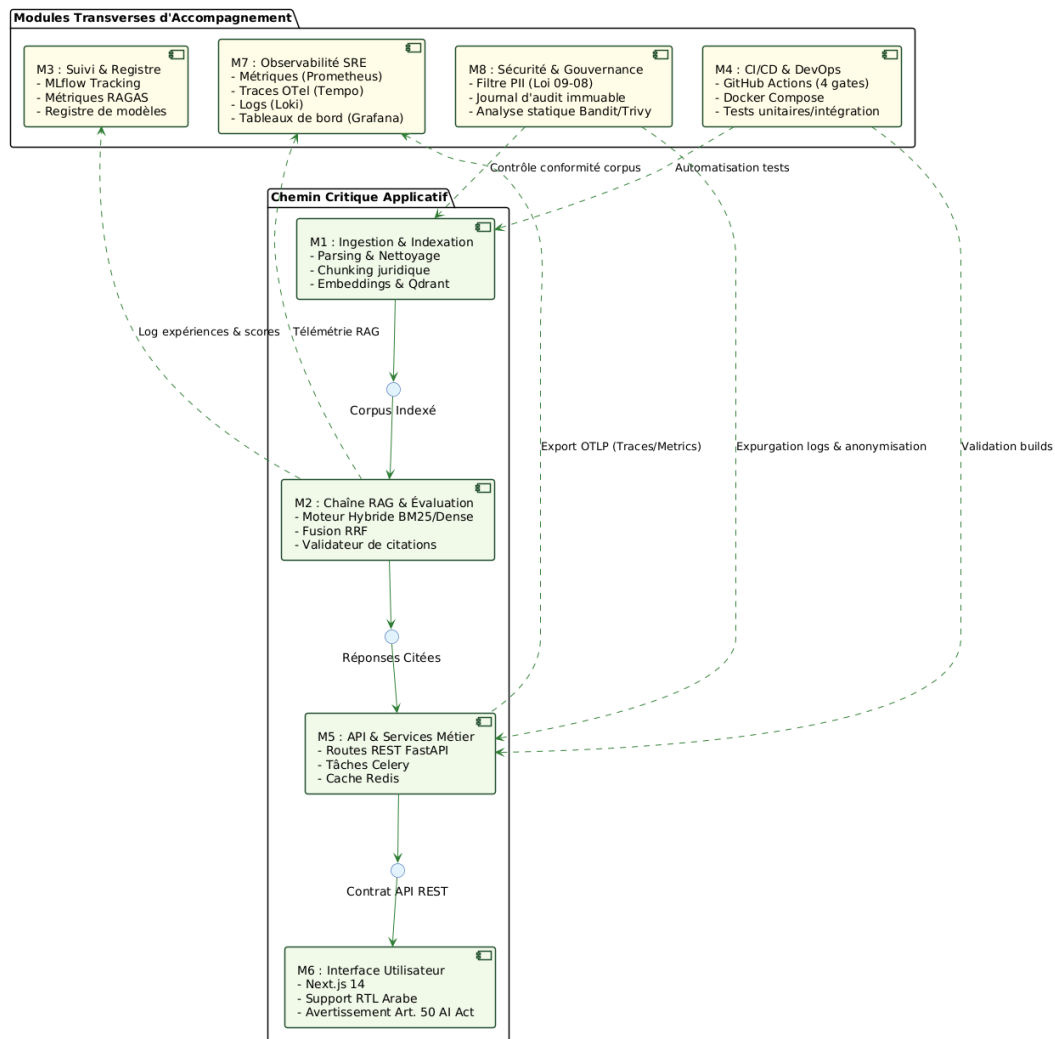


FIGURE 5.1 – Diagramme de composants. Le cadre central isole le chemin critique (M1 → M2 → M5 → M6), relié par trois interfaces fournies : le corpus indexé, les réponses citées et le contrat API. Le cadre supérieur regroupe les quatre modules transverses, dont les relations de dépendance sont figurées en pointillés.

5.2 Les trois interfaces du chemin critique

Le diagramme fait apparaître un élément déterminant pour l'organisation du travail en équipe : les quatre modules du chemin critique ne communiquent pas directement entre eux, mais à travers **trois interfaces explicitement nommées**.

Interface	Entre	Ce qu'elle garantit
Corpus indexé	M1 → M2	M2 n'a pas à connaître le format des documents sources ni la logique d'OCR
Réponses citées	M2 → M5	M5 n'a pas à connaître la stratégie de recherche ni le modèle employé
Contrat API REST	M5 → M6	M6 n'a pas à connaître la chaîne RAG, seulement la forme des réponses

TABLE 5.1 – Les trois interfaces qui découplent les modules du chemin critique.

Ce découplage a une conséquence pratique directe, confirmée par le déroulement du projet : chaque responsable de module a pu développer contre une interface stable plutôt que contre l'implémentation mouvante de son voisin. C'est aussi ce qui explique que le module M6 ait pu être développé alors que M5 n'était pas encore entièrement déployé — il suffisait que le *contrat* soit défini, non que le service soit opérationnel.

5.3 Pourquoi ce séquençement

Le choix de ce chemin critique n'est pas arbitraire : une réponse juridique ne peut être fiable que si le corpus qui l'alimente est propre (M1), si la chaîne de récupération et de génération est vérifiable (M2), si cette chaîne est exposée de façon sécurisée (M5), et si l'utilisateur final peut consulter et vérifier cette réponse (M6).

Les quatre modules transverses ne produisent aucune réponse juridique par eux-mêmes : ils garantissent que ce chemin reste *mesurable* (M3), *reproductible* (M4), *surveillable* (M7) et *conforme* (M8). C'est cette séparation qui permet, dans le chapitre 6, de documenter séparément ce qui a été prouvé sur le chemin critique lui-même et ce qui a été prouvé sur l'outillage qui l'entoure.

5.4 Pile technologique par couche

Couche	Module	Technologies principales
Données & ingestion	M1	Tesseract OCR (FR/AR), DVC, Great Expectations
Recherche & génération	M2	Qdrant, BM25 + recherche dense, fusion RRF, prompts versionnés
Suivi & registre	M3	MLflow, RAGAS
Plateforme & déploiement	M4	Docker, Docker Compose, GitHub Actions, Ruff, Bandit, pip-audit
API & service	M5	FastAPI, JWT/OAuth2, Redis, Celery, Locust (tests de charge)
Interface	M6	React/Next.js, TypeScript, Tailwind, Zustand, i18n FR/AR
Observabilité	M7	Prometheus, Grafana, Loki, Tempo, OpenTelemetry
Sécurité & conformité	M8	Bandit, pip-audit, scanner de secrets, journal d'audit HMAC

TABLE 5.2 – Correspondance entre couches d'architecture, modules responsables et technologies effectivement mises en œuvre et testées.

Cette organisation en couches, une fois posée, permet d'aborder chaque module indépendamment tout en gardant à l'esprit sa place dans l'ensemble : c'est l'objet du chapitre suivant.

Chapitre 6

Détail des modules

Chaque module est présenté selon la même grille de lecture : ce qui a été *réalisé et vérifié*, ce qui reste une *limite assumée* accompagnée de sa justification, et les *dépendances* qui le relie au reste du système. Cette symétrie de présentation est volontaire : elle permet de comparer directement le niveau de maturité de chaque brique sans effet de manche.

6.1 M1 — Ingestion et corpus juridique

Responsable : Douae Moussaoui

Le module M1 constitue le point d'entrée de tout le système : c'est lui qui transforme des documents juridiques bruts, potentiellement scannés, en un corpus indexable et gouverné.

✓ Réalisé et vérifié

- **Extraction et nettoyage** multi-format, avec repli sur OCR (Tesseract, français et arabe) lorsque le document source ne comporte pas de texte natif.
- **Anonymisation intégrée au pipeline**, effectuée entre le nettoyage et l'écriture — jamais après coup, ce qui évite qu'une donnée personnelle ne transite, même temporairement, par un fichier non protégé.
- **Déduplication** par empreinte SHA-256 et segmentation du texte, exportées dans un format structuré (`segments.jsonl`) consommable par le module M2.
- **Qualité en intégration continue** à deux niveaux : une porte bloquante par document, puis une porte bloquante sur l'ensemble du jeu de données via Great Expectations.
- **Gouvernance automatisée** : le pipeline DVC se termine par la génération automatique d'une fiche de données (*Data Card*), et la commande `dvc repro` permet de rejouer la chaîne complète de façon reproductible — vérifié à chaque contribution.

Limite assumée — et pourquoi

- Le corpus utilisé aujourd’hui est **entièrement synthétique**. Le connecteur vers la source légale réelle est présent dans le code mais lève volontairement une exception : publier ce connecteur sans validation juridique préalable du périmètre exposerait le projet à collecter des données réelles hors cadre légal validé.
- La correction orthographique n’est appliquée qu’au texte issu de l’OCR, via un lexique fermé, afin d’éviter la réécriture silencieuse d’un texte juridique par un correcteur généraliste — une modification non tracée d’un texte de loi serait plus grave qu’une faute d’OCR non corrigée.
- Le texte arabe est laissé strictement intact par ce correcteur : l’OCR arabe relève d’une nature différente (absence de casse, ligature) qui rendrait un correcteur pensé pour le français contre-productif.
- La validation du corpus arabe et le durcissement des règles d’anonymisation dépendent de travaux portés par M8 (le détecteur actuel, fondé sur des règles, affiche un rappel jugé encore insuffisant).

Dépendances : M1 attend de M2 la définition du périmètre de juridiction couvert, et de M8 la validation du corpus arabe ainsi que le durcissement des règles d’anonymisation.

6.2 M2 — Chaîne RAG vérifiable

Responsable : Imane Ibnchakroune

M2 est le cœur algorithmique du système : c’est ce module qui transforme une question en réponse citée, en s’interdisant explicitement d’inventer du contenu hors du corpus récupéré. Les fondements théoriques de cette chaîne sont exposés en §2.3.

✓ Réalisé et vérifié

- **Indexation ciblée** : découpage reproductible du corpus et indexation dans Qdrant, avec suppression propre par identifiant de document.
- **Recherche hybride** : combinaison BM25 (lexicale) et recherche dense (sémantique), fusionnées par la méthode RRF selon l’équation (2.3), avec filtrage sur métadonnées.
- **Génération sous contrainte** : compression extractive du contexte récupéré, *sans ajout de contenu*, avec des prompts versionnés pour garantir la traçabilité des évolutions du comportement du modèle.
- **Garde-fou structurel** : le système refuse de répondre en dehors de son périmètre déclaré, et valide chaque citation en exigeant une correspondance exacte avec le contexte effectivement récupéré — une citation qui ne peut pas être retrouvée mot pour mot dans le corpus est rejetée.
- L’ensemble des tests M2, le test de fumée Qdrant en local et le pipeline complet avec un générateur de test passent sur la branche principale de développement.

Décision de gouvernance

Fine-tuning paramétrique (LoRA / QLoRA) : retiré du périmètre v0.2.0. Cette décision a été actée par écrit et non laissée en suspens. L'infrastructure de préparation existe déjà côté M2, mais deux conditions préalables font défaut : aucun jeu de données question-réponse juridique annoté n'est disponible, et aucun modèle de base n'a été validé pour cet usage. La justification théorique de ce choix est développée en §2.4 ; entraîner un modèle sur des données non validées aurait produit un faux sentiment de progrès sans bénéfice mesurable.

Limite assumée — et pourquoi

- L'évaluation de la qualité de recherche reste bloquée par une dépendance externe : elle nécessite un jeu de pertinence annoté par des experts juridiques, qui n'existe pas encore.
- Les poids réels des modèles de production (BGE-M3, module de réordonnancement, grand modèle de langage) ne sont pas validés localement : le test de bout en bout avec un véritable modèle de langage n'est pas encore validé.
- Le traitement des réponses structurées en JSON est testé avec un pipeline Transformers, mais n'a pas encore été prouvé avec un modèle de langage de taille réduite réel.
- M2 se limite délibérément à fournir une interface Python injectable : la mise en cache et l'exposition via API sont déléguées à M5, le suivi des expériences à M3, l'affichage à M6.

Dépendances : M2 attend une validation experte externe pour l'évaluation du retrieval ; il délègue explicitement à M3, M5 et M6 les responsabilités hors de son périmètre fonctionnel.

6.3 M3 — Suivi d'expériences et registre de modèles

Responsable : Amal El Guerdani

M3 assure la mémoire du projet : chaque expérience de modélisation est tracée, et aucun modèle ne peut être promu sans passer par des contrôles de gouvernance explicites.

✓ Réalisé et vérifié

- **Suivi centralisé** via MLflow, intégré au pipeline M2 par un mécanisme dédié (*TrackingHook*).
- **Cadre d'évaluation** RAGAS [9] opérationnel, piloté par un exécuteur d'évaluation dédié.
- **Registre de modèles gouverné** : contrôles appliqués avant toute promotion, avec détection de régression.
- **Traçabilité stricte** : chaque fiche de modèle (*Model Card*) est rattachée à une version précise du jeu de données.
- 41 tests automatisés passent pour ce module en version 0.2.0, avec une intégration vérifiée sur une exécution MLflow réelle (et non simulée).

Limite assumée — et pourquoi

- Le **benchmark expert** — une évaluation de type « LLM-as-a-judge » sur 1200 questions juridiques annotées par des spécialistes — n'est pas réalisé. Le test d'intégration actuel valide que le pipeline d'évaluation fonctionne techniquement, mais ne constitue en aucun cas un banc d'essai juridique expert : la distinction est assumée explicitement pour ne pas laisser croire à une validation métier qui n'a pas eu lieu.

Décision de gouvernance

Trois règles de gouvernance appliquées dès la conception : (1) aucune question, réponse ou contexte juridique brut n'est jamais enregistré dans MLflow ; (2) aucun corpus artificiel n'est présenté comme une annotation experte ; (3) une mention de limitation est générée automatiquement et systématiquement dans chaque fiche de modèle.

Dépendances : M3 reçoit les résultats d'exécution de M2 et fournit la traçabilité nécessaire aux décisions de mise en production, sans dépendance bloquante externe identifiée à ce stade.

6.4 M4 — DevOps et ingénierie de plateforme

Responsable : Salma El Ouarrate

M4 fournit le socle d'exécution et d'automatisation sur lequel reposent tous les autres modules : c'est lui qui garantit qu'une contribution est testée, construite et publiée de façon reproductible.

✓ Réalisé et vérifié

- **Intégration continue** via GitHub Actions, déclenchée à chaque poussée de code et à chaque proposition de fusion.
- **Livraison continue** : toute fusion sur la branche principale déclenche une construction d'image Docker, publiée automatiquement sur un registre de conteneurs.
- **Conteneurisation durcie** : construction Docker multi-étapes, environnement Python 3.11, exécution sous un utilisateur non privilégié, OCR Tesseract français et arabe embarqué.

Implémenté et validé	Hors périmètre — différé
Docker & Docker Compose	Orchestration Kubernetes / Helm
CI/CD — GitHub Actions	Infrastructure as Code (Terraform)
Analyse statique — Ruff 0.16.7 + Pytest	Déploiement progressif (Canary)
Analyse de sécurité — Bandit + pip-audit	Pipeline de ré-entraînement automatique
Documentation — MkDocs, validation du site statique	

TABLE 6.1 – Périmètre explicitement implémenté contre périmètre explicitement différé pour M4.

Décision de gouvernance

Justification du périmètre différé : la priorité a été donnée à la reproductibilité de base plutôt qu'à une orchestration avancée. Mettre en place une orchestration complexe (Kubernetes, déploiement progressif) sans charge réelle à gérer produirait une configuration non testée en conditions réelles — c'est-à-dire une fausse impression de robustesse, et non une garantie de fonctionnement.

Dépendances : M4 ne dépend d'aucun autre module pour fonctionner ; en retour, M5 et M7 dépendent de lui respectivement pour la conteneurisation des composants Redis/Celery et pour la gestion centralisée des secrets.

6.5 M5 — API et couche de service

Responsable : Nouhaila Fadli

M5 expose le moteur RAG de M2 au reste du système de façon sécurisée et performante : c'est le seul point d'entrée autorisé vers la logique métier.

✓ Réalisé et vérifié

- **Intégration réelle** : les points d'entrée de l'API sont connectés au véritable moteur RAG, sans aucune simulation de réponse.
- **Sécurité** : authentification JWT/OAuth2 protégeant l'ensemble des routes sensibles.
- **Performance** : mise en cache via Redis et traitement asynchrone des documents via Celery.
- **Confidentialité par conception** : la question juridique de l'utilisateur est systématiquement transmise dans le corps de la requête HTTP, jamais dans l'URL — une URL peut être journalisée, mise en cache ou partagée sans que son contenu soit perçu comme sensible, contrairement au corps d'une requête.
- Les tests `pytest` passent intégralement pour la version 0.2.0, et un test de charge (Locust) a été validé et documenté.

Limite assumée — et pourquoi

- Le déploiement conteneurisé de Redis et Celery, ainsi que la gestion centralisée des secrets, dépendent de la conteneurisation finale livrée par M4.
- Le flux de réponse en continu est actuellement **simulé** : le mécanisme *Server-Sent Events* est bien démontré, mais la réponse est en réalité découpée artificiellement après une génération complète, plutôt que diffusée jeton par jeton depuis un flux réel.
- Le système reste **monutilisateur** : il s'appuie sur un utilisateur de test unique, sans base de données multi-utilisateurs.

Décision de gouvernance

Profondeur avant largeur. Le choix a été fait d'implémenter et de prouver exhaustivement quatre mécanismes fondamentaux (intégration réelle, sécurité, cache, confidentialité) plutôt que d'ébaucher cinq fonctionnalités supplémentaires sans les rendre robustes.

Dépendances : M5 attend de M4 la conteneurisation de ses composants et la gestion des secrets ; il fournit en retour à M6 le contrat d'interface nécessaire à l'affichage des réponses.

6.6 M6 — Frontend et expérience utilisateur

Responsable : Oumaima Jeraïdi

M6 est la seule partie du système directement visible par l'utilisateur final : son rôle est de rendre la réponse juridique lisible, vérifiable et accessible, dans les deux langues du projet.

✓ Réalisé et vérifié

- **Architecture** : quatre routes principales et dix-huit composants, construits sur des jetons de conception Tailwind cohérents.
- **Internationalisation stricte** : parité complète entre le français et l'arabe sur quatre-vingt-douze clés de traduction, avec bascule automatique de la direction du texte (*RTL*) pour l'arabe.
- **Couche frontière dédiée** : un module d'adaptation traduit le contrat d'interface de M5 vers le format interne du frontend, isolant ainsi la logique métier de la présentation — un changement côté API n'impose pas de réécrire l'ensemble de l'interface.
- **Gestion d'état** centralisée via un magasin Zustand reliant la barre latérale et l'historique des échanges.
- Vérifié par analyse statique du code : zéro attribut d'accessibilité (`aria-label`) codé en dur dans les 2159 lignes de code TypeScript/TSX du projet — tous proviennent du système de traduction.

Limite assumée — et pourquoi

- La consommation réelle de l'API et le téléversement effectif de documents restent bloqués : cette logique attend le déploiement complet de M5.
- Le bouton d'export PDF est **visuellement présent mais non fonctionnel** : aucune dépendance de génération de PDF n'existe dans le projet. Ce choix est assumé et documenté plutôt que masqué.
- **Aucun audit d'accessibilité chiffré** (référentiel RGAA ou norme WCAG 2.1 niveau AA [14]) n'a été mesuré. Les attributs d'accessibilité sont bien traduits et la gestion du focus est en place, mais aucun pourcentage de conformité ne sera revendiqué sans avoir été effectivement mesuré.
- La route de documentation interne du système de conception a été retirée pour la présentation, par choix éditorial et non par contrainte technique.

Dépendances : M6 attend de M5 la disponibilité complète de l'API réelle pour lever ses deux principales limites fonctionnelles.

6.7 M7 — Observabilité SRE et ML

Responsable : Youssef El Alem

M7 répond à une question simple mais essentielle : si le système tombe en panne, l'équipe doit pouvoir en connaître la cause sans avoir à deviner. Ce module ne produit aucune réponse juridique ; il rend le reste du système observable. Les trois signaux qu'il exploite sont présentés en §2.6.

✓ Réalisé et vérifié

- **Métriques** : Prometheus expose neuf métriques applicatives et huit règles d'alerte, validées à l'aide d'un simulateur de charge strict — c'est-à-dire que le déclenchement effectif des alertes est vérifié, pas seulement leur définition syntaxique.
- **Journaux** : Loki centralise les journaux applicatifs avec une durée de rétention fixée à trois ans, une décision de gouvernance documentée (référence interne : décision A-4) et non un choix arbitraire.
- **Traces** : Tempo et OpenTelemetry conservent les traces de requêtes pendant trente jours pour l'analyse de latence.
- **Sécurité des traces** : le collecteur OTLP est configuré pour purger les données personnelles *avant* le stockage. Vérifié par quarante tests automatisés : sur une trace de test comportant onze attributs sensibles, le collecteur n'en laisse passer que deux — une valeur mesurée, non estimée.

Limite assumée — et pourquoi

- Le code d'instrumentation Prometheus est prêt, mais la bascule réseau réelle dépend de la conteneurisation finale livrée par M4.
- La détection de dérive via Evidently exige un volume de trafic réel entre M2 et M5 pour être statistiquement significative : sans trafic réel, le résultat ne serait que du bruit.
- Le choix entre échantillonnage en tête et en fin de trace a été **délibérément laissé non configuré**. Trancher sans charge réelle serait prématuré, même si le *tail sampling* est d'ores et déjà identifié comme la cible pertinente pour isoler les traces lentes.
- Le service d'alerte fonctionne, mais ses notifications sont coupées : elles attendent la gestion centralisée des secrets portée par M4.

Dépendances : M7 dépend de M4 pour la mise en réseau réelle et la gestion des secrets de notification, et de M2/M5 pour disposer d'un trafic réel exploitable par la détection de dérive.

6.8 M8 — Sécurité, gouvernance et conformité

Responsable : Taha Kachmar

M8 est le module transverse par excellence : il ne produit pas de fonctionnalité visible, mais conditionne la légitimité de l'ensemble du système au regard de la protection des données personnelles et de la réglementation applicable à l'intelligence artificielle.

✓ Réalisé et vérifié

- **Placement de l'anonymisation** : appliquée entre le nettoyage et l'écriture du corpus (voir §6.1) — le dernier point où retirer une personne du texte reste une simple modification de texte, et non une reconstruction complète de l'index.
- **Journal d'audit conforme** : les actions utilisateur sont journalisées de façon pseudonymisée (par HMAC), le système refusant explicitement d'écrire une adresse électronique, une adresse IP ou un jeton d'authentification en clair, et refusant d'écrire tout événement en l'absence de clé de pseudonymisation.
- **Qualification réglementaire** : au regard du règlement européen sur l'intelligence artificielle [13], l'usage envisagé par les juridictions est explicitement exclu du périmètre (décision interne A-3), ce qui place le système hors de la catégorie « haut risque ». Seule l'obligation de transparence de l'article 50 demeure applicable.
- **Contrôles bloquants en intégration continue** : quatre portes obligatoires. Le scanner de secrets reconnaît les jetons par leur forme, indépendamment de la casse : sur un banc de test, il détectait 0 secret sur 7 avant sa mise en place, contre 7 sur 7 après, sans aucun faux positif.
- Cent tests automatisés dédiés à ce module sont exécutés à chaque publication de version.

Banc de 39 cas	Rappel (données détectées)	Domage (texte masqué à tort)
Français	89,5 %	0 %
Arabe	71,4 %	0 %
Ensemble	85,7 %	0 %

TABLE 6.2 – Résultats du banc de test d'anonymisation. Le domage nul confirme qu'aucune phrase juridique n'est masquée à tort ; l'écart de rappel entre langues s'explique par l'absence de majuscules en arabe, qui prive le détecteur d'un indice fort utilisé en français.

Registre des écarts de conformité — version 1.7 : cinq fiches, quinze écarts fermés, cinq écarts encore ouverts.

- **E-01** — La détection s'appuie sur des règles (français et arabe) et non sur la reconnaissance d'entités nommées : un nom jamais introduit par une civilité peut échapper au masquage. Point bloquant avant tout passage à un corpus réel. *(M8)*
- **E-06** — Le chiffrement au repos n'est pas documenté par l'hébergeur ; le corpus étant entièrement synthétique aujourd'hui, l'impact réel est nul pour l'instant. *(M8 + M1)*
- **E-11** — Le champ `SourceType` mélange catégories de document et canaux de collecte, ce qui nuit à la lisibilité du registre. *(M8 + M1)*
- **E-13** — Le point d'entrée d'analyse de documents transmettait le nom du document dans l'URL ; déjà corrigé et retiré des traces de télémétrie (voir §6.7). *(M5)*
- **E-14** — Le journal d'audit est livré et sa rétention configurée, mais l'API ne l'appelle pas encore : aucun événement n'est effectivement écrit à ce jour. *(M5 + M8)*

Décision de gouvernance

Corrections assumées publiquement. Deux erreurs antérieures du registre ont été corrigées par une note datée plutôt que silencieusement : (1) l'analyse d'impact affirmait que la propagation des noms était indépendante de la langue — vérifié en conditions réelles, aucun nom arabe n'était en fait propagé ; (2) le registre associé à la version 0.2.0 déclarait deux écarts encore ouverts alors que les rétentions Loki et Tempo correspondantes étaient déjà configurées, et il omettait l'écart E-14.

Dépendances : M8 conditionne la validation du corpus arabe et le durcissement de l'anonymisation pour M1 ; il documente des écarts touchant directement M5 (E-13, E-14).

Chapitre 7

Résultats et démonstration

Ce chapitre présente le système en fonctionnement. Les captures d’écran qui suivent ont toutes été réalisées sur une instance effectivement déployée, et non reconstituées : elles constituent la preuve visuelle des affirmations formulées au chapitre 6.

7.1 Interface utilisateur

7.1.1 Tableau de bord

La figure 7.1 présente le tableau de bord, point d’entrée de l’application. Il agrège les indicateurs d’usage et l’état des documents analysés.

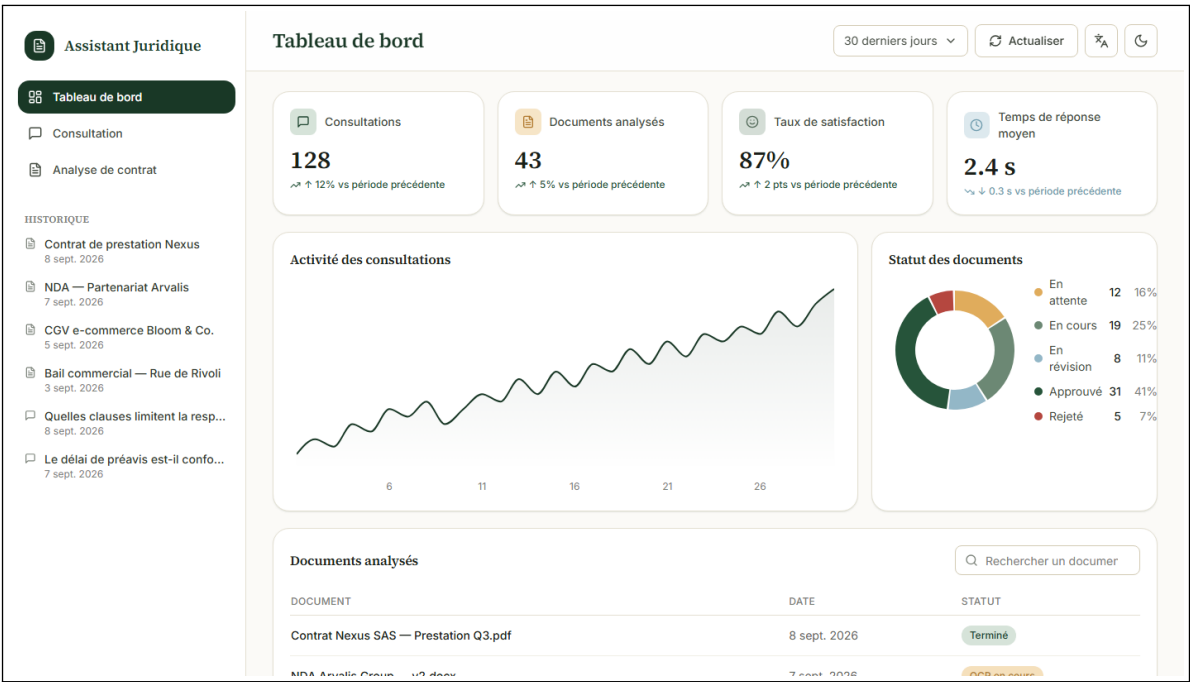


FIGURE 7.1 – Tableau de bord de l’assistant juridique. On y distingue la barre latérale de navigation, l’historique des consultations, les indicateurs agrégés et le tableau des documents analysés. Le sélecteur de langue et le bascule de thème figurent en haut à droite.

7.1.2 Consultation juridique

La figure 7.2 présente l’écran de consultation, où l’utilisateur formule sa question. Un élément mérite l’attention : la mention placée sous le champ de saisie.

7.2 Supervision et observabilité

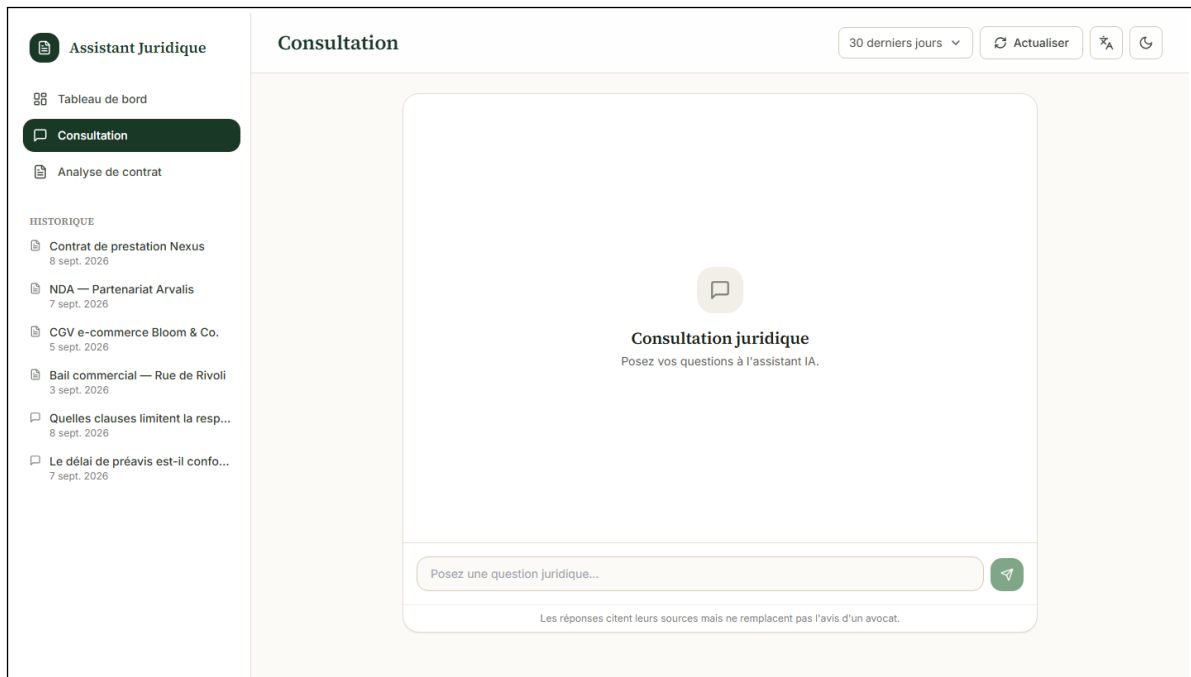


FIGURE 7.2 – Écran de consultation. La mention « *Les réponses citent leurs sources mais ne remplacent pas l'avis d'un avocat* » n'est pas une précaution rédactionnelle : elle matérialise l'obligation de transparence de l'article 50 du règlement européen sur l'intelligence artificielle, identifiée par le module M8 (§6.8).

7.2.1 Tableau de bord Grafana

La figure 7.3 présente le tableau de bord principal du module M7, alimenté par des données réelles issues du simulateur de charge.

Cette capture illustre concrètement l'intérêt de la séparation par domaines : une dégradation de la latence globale visible en haut peut être immédiatement rapportée, en bas, soit à la phase de récupération, soit à la génération. C'est la traduction opérationnelle du principe d'observabilité exposé en §2.6.

7.2.2 Règles d'alerte

La figure 7.4 présente les règles d'alerte effectivement chargées dans Prometheus, regroupées par domaine fonctionnel.

Le regroupement par domaine n'est pas cosmétique : il permet, lors d'un incident, de distinguer immédiatement une dégradation de performance (*latence*) d'une indisponibilité (*disponibilité*) ou d'un défaut applicatif (*erreurs*), et donc d'orienter le diagnostic vers le bon module.

7.2.3 Cibles supervisées

La figure 7.5 confirme que la supervision est effectivement connectée à sa cible applicative.



FIGURE 7.3 – Tableau de bord Grafana du module M7. Le domaine API (haut) expose le débit, le taux d’erreur, la latence au percentile 95 et le taux de cache ; le domaine *retriever* et modèle de langage (bas) expose la latence de récupération comparée à son objectif de 120 ms (ligne pointillée rouge), le nombre de passages remontés et la consommation de jetons.

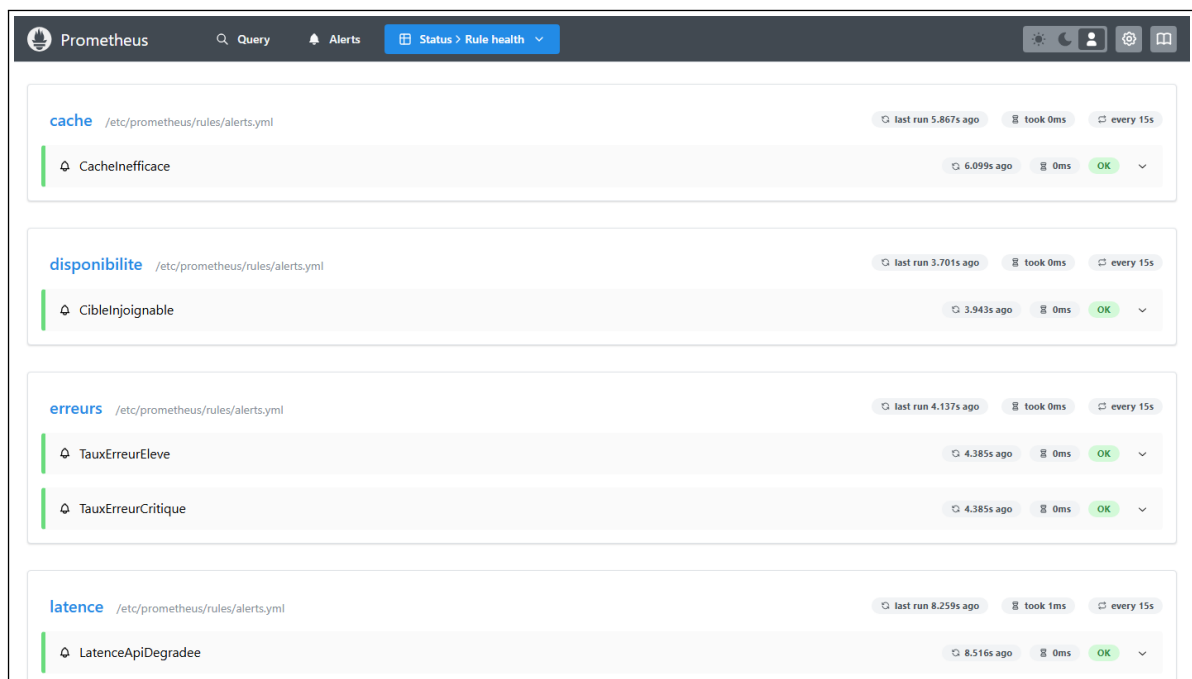
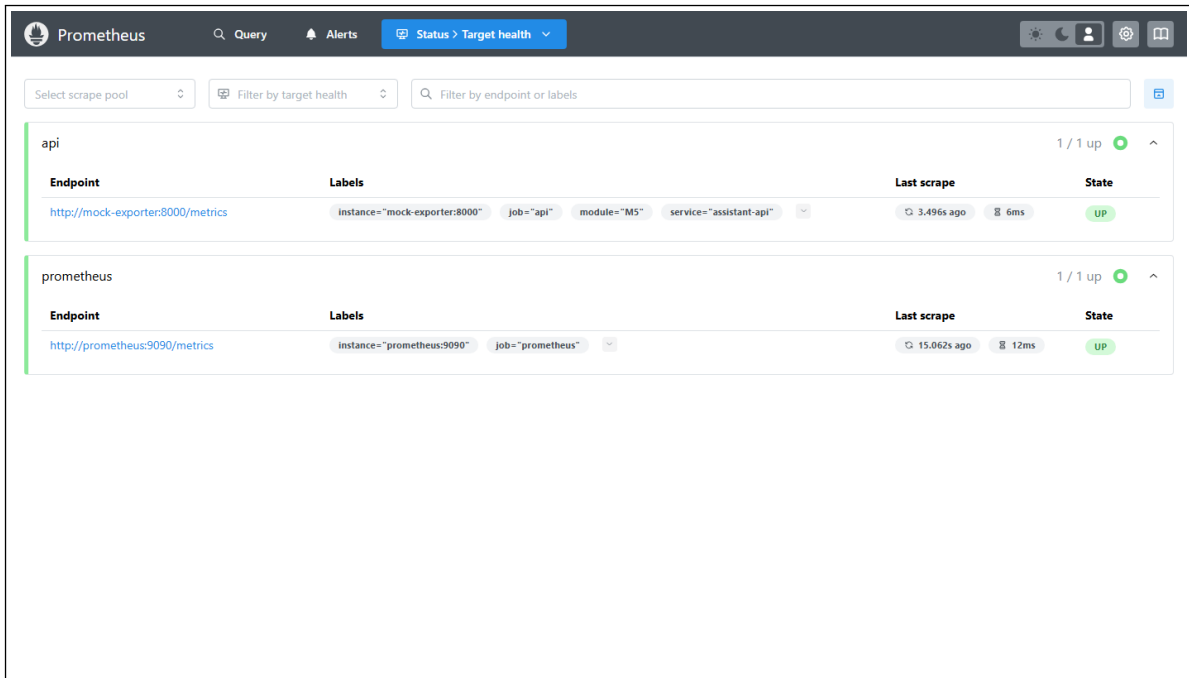


FIGURE 7.4 – Règles d’alerte du module M7, regroupées par domaine (cache, disponibilité, erreurs, latence). Chaque règle est évaluée toutes les quinze secondes ; l’état « OK » indique que la condition d’alerte n’est pas remplie au moment de la capture.



The screenshot shows the Prometheus 'Status > Target health' page. It displays two targets with their endpoints, labels, last scrape times, and states.

Target	Endpoint	Labels	Last scrape	State
api	http://mock-exporter:8000/metrics	instance="mock-exporter:8000" job="api" module="M5" service="assistant-api"	3.496s ago 8 6ms	UP
prometheus	http://prometheus:9090/metrics	instance="prometheus:9090" job="prometheus"	15.062s ago 8 12ms	UP

FIGURE 7.5 – Cibles de collecte. La cible `api` porte les étiquettes `module="M5"` et `service="assistant-api"` : cet étiquetage permet de rattacher chaque métrique au module responsable, ce qui est indispensable dans un système où huit modules coexistent.

7.2.4 Traçage distribué

La figure 7.6 présente une trace complète dans Tempo, c'est-à-dire le parcours détaillé d'une requête à travers les composants du système.

7.3 Synthèse des résultats obtenus

Le tableau 7.1 rassemble les indicateurs vérifiables du projet, tous établis par des mesures automatisées et non par estimation.

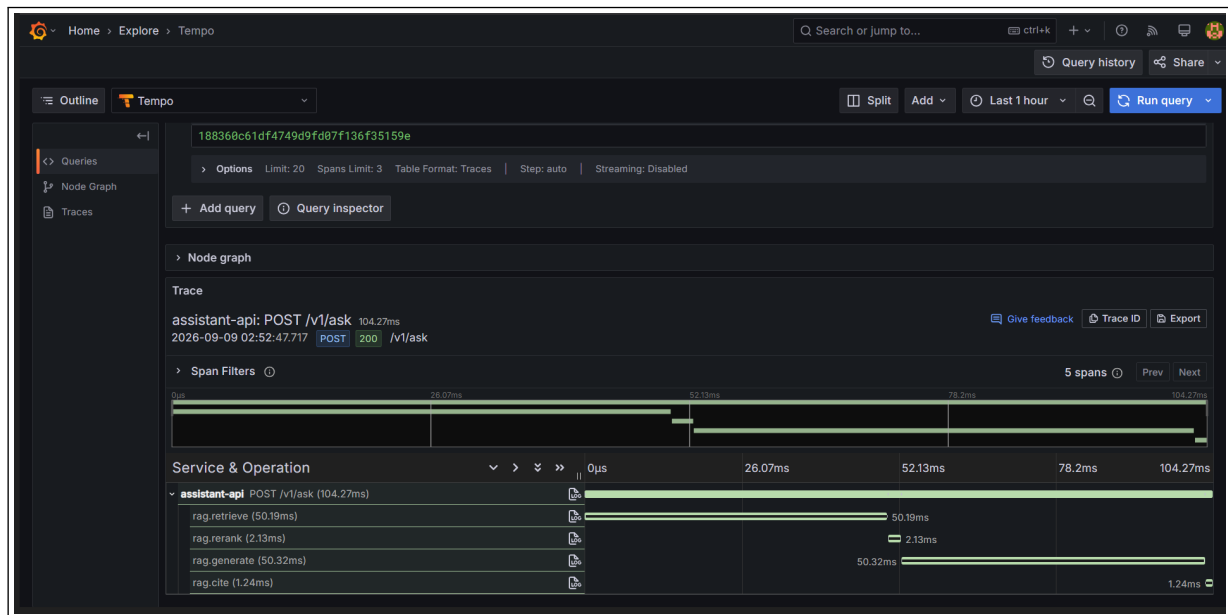


FIGURE 7.6 – Trace d’une requête à travers la chaîne. La décomposition en segments permet d’attribuer précisément le temps passé à chaque étape — récupération, réordonnancement, génération — et donc d’identifier le goulot d’étranglement réel plutôt que supposé.

Indicateur	Valeur	Mode d’obtention
Modules livrés	8 / 8	Chemin critique complet et modules transverses
Tests automatisés (M3)	41	Exécution en intégration continue
Tests automatisés (M7)	40	Vérification de l’expurgation du collecteur
Tests automatisés (M8)	100	Exécutés à chaque publication de version
Métriques exposées	9	Point d’entrée Prometheus
Règles d’alerte	8	Validées par simulateur de charge
Services conteneurisés	8	Pile d’observabilité, une commande de démarrage
Attributs sensibles purgés	9 / 11	Banc de trace piégée (2 conservés, non sensibles)
Rappel d’anonymisation	85,7 %	Banc de 39 cas bilingues
Domage d’anonymisation	0 %	Aucun texte juridique masqué à tort
Détection de secrets	7 / 7	Banc de test, sans faux positif
Clés de traduction FR/AR	92	Parité vérifiée par analyse statique
Contributions au dépôt	135	Sur sept semaines

TABLE 7.1 – Synthèse des indicateurs du projet. Chaque valeur est rattachée à son mode d’obtention : aucun de ces chiffres ne résulte d’une estimation.

Limite assumée — et pourquoi

Un indicateur est volontairement absent de ce tableau : la **qualité des réponses juridiques**. Elle ne peut être mesurée qu’avec un jeu de questions annoté par des experts du droit marocain, qui n’existe pas à ce jour (§6.3). Publier un score de qualité fondé sur un corpus synthétique reviendrait à présenter une validation technique comme une validation métier : c’est précisément ce que la méthodologie de ce projet s’interdit.

Chapitre 8

Décisions transverses et gouvernance

Au-delà des décisions propres à chaque module, deux mécanismes transverses structurent la gouvernance technique du projet dans son ensemble.

8.1 Carte des dépendances inter-modules

Le tableau 8.1 rassemble, module par module, les dépendances bloquantes explicitement identifiées au cours du développement. Cette carte permet de lire d'un coup d'œil pourquoi certaines limites signalées au chapitre 6 ne relèvent pas d'un retard de l'équipe responsable, mais d'un enchaînement logique entre modules.

Module	Attend de...
M1	M2 (périmètre de juridiction) — M8 (validation du corpus arabe, règles d'anonymisation)
M2	Validation experte externe (jeu de pertinence annoté)
M3	Aucune dépendance bloquante identifiée
M4	Aucune dépendance bloquante identifiée
M5	M4 (conteneurisation Redis/Celery, gestion des secrets)
M6	M5 (API réelle disponible, téléversement documentaire)
M7	M4 (réseau, secrets) — M2/M5 (trafic réel pour la détection de dérive)
M8	Aucune dépendance bloquante — documente des écarts touchant M1 et M5

TABLE 8.1 – Carte de dépendances inter-modules. Trois modules seulement (M3, M4, M8) sont autonomes ; ce sont précisément ceux dont les livrables sont les plus complets.

Cette carte fait apparaître une régularité : les modules autonomes (M3, M4, M8) sont ceux qui présentent le moins de limites au chapitre 6, tandis que les modules les plus contraints (M6, M7) concentrent les limites fonctionnelles. La corrélation n'est pas fortuite : elle confirme que les limites documentées relèvent majoritairement de l'ordonnancement des dépendances, et non de la difficulté intrinsèque des travaux.

8.2 Le point de convergence : le module M4

Une lecture attentive du tableau 8.1 révèle que le module M4 apparaît comme prérequis de M5 et de M7, lesquels conditionnent à leur tour M6. La conteneurisation finale constitue donc le **point de convergence** du projet : sa livraison lèverait simultanément quatre des limites documentées.

Décision de gouvernance

Conséquence pour la suite. Cette observation détermine directement la priorisation de la version 0.3.0 présentée au chapitre 9 : il ne s'agit pas de traiter les limites par ordre de gravité apparente, mais par ordre de *levée de dépendances*. Un travail sur M4 débloque quatre limites ; un travail équivalent sur M6 n'en débloque aucune.

8.3 Une décision de périmètre valant pour l'ensemble du projet

La décision la plus structurante du cycle 0.2.0 reste le retrait du fine-tuning paramétrique (§6.2), dont la justification théorique est développée en §2.4. Elle a été actée par écrit, tracée par un identifiant de suivi, et communiquée à l'ensemble de l'équipe plutôt que laissée à l'appréciation individuelle d'un seul module.

C'est ce mode de décision — explicite, daté, traçable — qui a ensuite été repris pour documenter chaque limite assumée présentée dans ce rapport. Il constitue, rétrospectivement, l'apport méthodologique le plus transposable du projet : une limite tracée et justifiée reste exploitable par l'équipe suivante ; une limite passée sous silence redevient un piège.

8.4 Bilan de la gouvernance des contributions

Le tableau 8.2 synthétise les mécanismes de gouvernance effectivement appliqués pendant les sept semaines du projet.

Mécanisme	Effet obtenu	Appliqué
Propriétaires de code (CODEOWNERS)	Aucun module modifié sans l'accord de son équipe responsable	Systématique
Invalidation des revues obsolètes	Aucune contribution approuvée puis modifiée avant fusion	Systématique
Quatre contrôles bloquants	Aucune fusion sans tests, analyse de sécurité et documentation valides	Systématique
Épinglage des versions d'outils	Résultats d'analyse statique reproductibles entre machines	Depuis S6
Registre des écarts de conformité	Écarts publiés et datés plutôt que corrigés silencieusement	Depuis S3

TABLE 8.2 – Mécanismes de gouvernance des contributions et effets constatés.

Chapitre 9

Conclusion générale et perspectives

9.1 Bilan du travail réalisé

Ce rapport a présenté la conception, la mise en œuvre et la gouvernance d'un assistant juridique intelligent fondé sur l'architecture RAG, industrialisé selon les pratiques MLOps. Trois constats se dégagent de ce travail.

Le chemin critique fonctionne réellement de bout en bout. Une question juridique soumise via l'interface (M6) est aujourd'hui traitée par une chaîne de récupération et de génération vérifiable (M2), à partir d'un corpus gouverné (M1), exposée par une interface de programmation sécurisée (M5). Ce résultat n'était pas garanti d'avance : il découle directement du choix méthodologique, exposé au chapitre 1, de privilégier la profondeur d'une chaîne complète plutôt que la largeur de fonctionnalités isolées.

Les modules transverses conditionnent la crédibilité du système. Le suivi et le registre (M3), la plateforme (M4), l'observabilité (M7) et la conformité (M8) ne sont pas des annexes du projet. C'est grâce à eux que chaque affirmation de ce rapport peut être rattachée à une preuve vérifiable — un test, un nombre de lignes analysées, un résultat de simulateur — plutôt qu'à une déclaration d'intention. Un prototype RAG sans ces quatre modules aurait pu être démontré en une semaine ; il n'aurait été ni reproductible, ni supervisable, ni conforme.

L'honnêteté sur les limites est une exigence d'ingénierie. C'est sans doute l'apport méthodologique le plus important de ce projet. Chaque limite documentée dans ce rapport — qu'il s'agisse d'un bouton visuellement présent mais inactif (§6.6), d'un échantillonnage volontairement non configuré (§6.7), ou d'écarts de conformité publiés plutôt que dissimulés (§6.8) — est accompagnée de sa justification technique. Cette discipline, appliquée uniformément aux huit modules, est ce qui permet à ce rapport de se lire non comme une liste de promesses, mais comme un état des lieux vérifiable.

9.2 Réponse à la problématique

La problématique posée au chapitre 1 interrogeait la possibilité de concevoir un assistant juridique garantissant *structurellement* la provenance de ses réponses, tout en respectant le cadre réglementaire.

La réponse apportée par ce travail est double.

Sur le plan de la **provenance**, la garantie n'est pas obtenue par la qualité du modèle de langage — sur laquelle on ne peut pas s'appuyer — mais par une contrainte *architecturale* : le modèle ne génère qu'à partir de passages récupérés, et chaque citation produite est revalidée contre ces

passages avant d’être affichée. Une citation introuvable mot pour mot est rejetée. La garantie est donc structurelle, au sens où elle ne dépend pas du comportement du modèle.

Sur le plan de la **conformité**, la réponse réside dans le *placement* des mécanismes plutôt que dans leur simple existence : l’anonymisation intervient avant l’écriture du corpus et non après ; l’expurgation de la télémétrie intervient avant le stockage et non au moment de la consultation ; la question de l’utilisateur transite par le corps de la requête et non par l’URL. Dans les trois cas, le mécanisme est placé au dernier point où la donnée peut encore être retirée simplement.

9.3 Limites du travail

Il serait malhonnête de conclure sans rappeler les deux limites majeures qui subsistent, déjà documentées mais qu’il convient de récapituler.

La première est l’**absence de corpus juridique réel validé**. Le système fonctionne aujourd’hui sur un corpus synthétique, ce qui suffit à démontrer la chaîne technique mais ne permet aucune conclusion sur la pertinence juridique des réponses. Le franchissement de cette étape est conditionné par la levée de l’écart E-01 (§6.8), c’est-à-dire par le renforcement du détecteur de données personnelles.

La seconde est l’**absence d’évaluation experte**. Aucun score de qualité juridique n’est revendiqué dans ce rapport, faute d’un jeu de questions annoté par des professionnels du droit. Cette limite n’est pas d’ordre technique : le cadre d’évaluation RAGAS est opérationnel et testé (§6.3) ; c’est la donnée de référence qui manque.

9.4 Perspectives

Le tableau 9.1 situe la version couverte par ce rapport dans la trajectoire du projet.

Version	Statut	Contenu
v0.1.0	Publiée le 26/08/2026	Socle initial du projet et premières briques par module.
v0.2.0	Publiée le 15/09/2026	Chaîne bout en bout fonctionnelle (M1→M2→M5→M6), gouvernance et observabilité mises en place, limites documentées module par module. <i>Objet de ce rapport.</i>
v0.3.0	Planifiée	Levée des dépendances bloquantes identifiées au chapitre 8, en commençant par le point de convergence M4.

TABLE 9.1 – Trajectoire de versions du projet.

9.4.1 Priorités à court terme

Conformément au raisonnement exposé au chapitre 8, les priorités sont ordonnées par *levée de dépendances* et non par gravité apparente :

1. **Achever la conteneurisation (M4)**, ce qui lève simultanément quatre limites : le déploiement de Redis et Celery, la gestion des secrets, la bascule réseau de l’instrumentation

et l'activation des notifications d'alerte.

2. **Renforcer le détecteur de données personnelles (M8, écart E-01)**, condition préalable à tout passage à un corpus réel.
3. **Brancher le journal d'audit à l'API (écart E-14)**, aujourd'hui livré mais jamais appelé.

9.4.2 Perspectives à moyen terme

Au-delà de ces travaux de consolidation, trois directions se dégagent :

- **Constituer un jeu d'évaluation annoté** en collaboration avec des professionnels du droit. C'est la condition d'une mesure de qualité crédible, et donc de toute comparaison ultérieure entre configurations.
- **Activer la détection de dérive** une fois un trafic réel disponible, puis configurer l'échantillonnage en fin de trace (*tail sampling*) sur la base des profils de latence observés.
- **Reconsidérer l'ajustement fin**, non comme alternative au RAG mais comme complément : une fois un jeu annoté disponible, un ajustement léger pourrait améliorer le *style* des réponses sans toucher à la garantie de provenance, qui resterait assurée par l'architecture.

9.5 Apport personnel et formation

Ce projet a confronté l'équipe à une réalité peu visible dans un cursus académique : dans un système d'apprentissage automatique en production, l'algorithme n'est pas la difficulté principale. Les briques algorithmiques mobilisées ici — BM25, recherche dense, fusion RRF — sont toutes documentées et disponibles. La difficulté réelle résidait dans leur assemblage sous contrainte : rendre le résultat reproductible, supervisable, conforme, et surtout *démontrable*.

C'est cette dernière exigence qui aura le plus durablement marqué la méthode de travail du groupe. Affirmer qu'une fonctionnalité marche engage peu ; devoir produire le test, la mesure ou la capture qui l'établit change entièrement la manière de concevoir et de rendre compte d'un travail d'ingénierie.

Bibliographie

- [1] LEWIS, P., PEREZ, E., PIKTUS, A. *et al.* « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] VASWANI, A., SHAZEER, N., PARMAR, N. *et al.* « Attention Is All You Need ». *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [3] ROBERTSON, S. et ZARAGOZA, H. « The Probabilistic Relevance Framework : BM25 and Beyond ». *Foundations and Trends in Information Retrieval*, vol. 3, n° 4, p. 333–389, 2009.
- [4] CORMACK, G. V., CLARKE, C. L. A. et BUETTCHER, S. « Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods ». *Proceedings of the 32nd International ACM SIGIR Conference*, p. 758–759, 2009.
- [5] KARPUKHIN, V., OĞUZ, B., MIN, S. *et al.* « Dense Passage Retrieval for Open-Domain Question Answering ». *Proceedings of EMNLP*, p. 6769–6781, 2020.
- [6] CHEN, J., XIAO, S., ZHANG, P. *et al.* « BGE M3-Embedding : Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation ». *arXiv preprint arXiv :2402.03216*, 2024.
- [7] HU, E. J., SHEN, Y., WALLIS, P. *et al.* « LoRA : Low-Rank Adaptation of Large Language Models ». *International Conference on Learning Representations (ICLR)*, 2022.
- [8] DETTMERS, T., PAGNONI, A., HOLTZMAN, A. et ZETTLEMOYER, L. « QLoRA : Efficient Finetuning of Quantized LLMs ». *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] ES, S., JAMES, J., ESPINOSA-ANKE, L. et SCHOCKAERT, S. « RAGAS : Automated Evaluation of Retrieval Augmented Generation ». *arXiv preprint arXiv :2309.15217*, 2023.
- [10] SCULLEY, D., HOLT, G., GOLOVIN, D. *et al.* « Hidden Technical Debt in Machine Learning Systems ». *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [11] BEYER, B., JONES, C., PETOFF, J. et MURPHY, N. R. *Site Reliability Engineering : How Google Runs Production Systems*. O'Reilly Media, 2016.
- [12] ROYAUME DU MAROC. *Loi n° 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel*. Bulletin officiel, 2009.
- [13] PARLEMENT EUROPÉEN ET CONSEIL DE L'UNION EUROPÉENNE. *Règlement (UE) 2024/1689 établissant des règles harmonisées concernant l'intelligence artificielle*. Journal officiel de l'Union européenne, 2024.
- [14] WORLD WIDE WEB CONSORTIUM (W3C). *Web Content Accessibility Guidelines (WCAG) 2.1*. Recommandation du W3C, juin 2018. <https://www.w3.org/TR/WCAG21/>
- [15] PROMETHEUS AUTHORS. *Prometheus — Monitoring System and Time Series Database, Documentation*. <https://prometheus.io/docs/>
- [16] OPENTELEMETRY AUTHORS. *OpenTelemetry Documentation — Traces, Metrics, Logs*. <https://opentelemetry.io/docs/>

- [17] QDRANT TEAM. *Qdrant — Vector Database Documentation*. <https://qdrant.tech/documentation/>
- [18] MLFLOW PROJECT. *MLflow Documentation — Tracking, Models, Model Registry*. <https://mlflow.org/docs/latest/>

Chapitre A

Répartition détaillée des responsabilités

Module	Responsable	Intitulé du rôle
M1	Douae Moussaoui	Data Engineer — responsable du pipeline de données
M2	Imane Ibnchakroune	ML / LLM Engineer — responsable de la modélisation
M3	Amal El Guerdani	MLOps Engineer — responsable de l'expérimentation
M4	Salma El Ouarrate	DevOps / Platform Engineer — responsable CI/CD
M5	Nouhaila Fadli	Backend Engineer — responsable API et service
M6	Oumaima Jeraidi	Frontend Engineer — responsable UI/UX
M7	Youssef El Alem	SRE / ML Observability — responsable supervision
M8	Taha Kachmar	Security & Compliance Officer — responsable gouvernance

TABLE A.1 – Répartition nominative des responsabilités. La coordination générale du projet est assurée par Youssef El Alem, en sus du module M7.

Chapitre B

Glossaire des termes techniques

Ancrage (*grounding*)

Fait de contraindre un modèle de langage à ne produire que des énoncés rattachables à une source fournie, par opposition à une génération libre.

Chunking (découpage)

Segmentation d'un document en fragments de taille contrôlée avant indexation. Le compromis porte sur la taille : trop court, le fragment perd son contexte ; trop long, il dilue le signal pertinent.

Dérive (*drift*)

Dégradation progressive des performances d'un système déployé, due à l'évolution des données entrantes (*data drift*) ou de la relation entre entrée et sortie attendue (*concept drift*), sans modification du code.

Hallucination

Production par un modèle de langage d'un énoncé plausible mais factuellement faux ou sans fondement dans une source réelle.

Observabilité

Capacité à diagnostiquer l'état interne d'un système à partir de ses signaux externes, y compris pour des pannes non anticipées. À distinguer de la simple surveillance, qui ne vérifie que des conditions connues d'avance.

Plongement (*embedding*)

Représentation d'un texte sous forme de vecteur numérique, construite de sorte que la proximité géométrique traduise la proximité sémantique.

Pseudonymisation

Remplacement d'un identifiant direct par un identifiant dérivé (ici par HMAC), de façon à permettre le recoupement d'événements sans exposer l'identité. À distinguer de l'anonymisation, qui est irréversible.

Rappel (*recall*)

Proportion des éléments à détecter qui ont effectivement été détectés. Pour un détecteur de données personnelles, un rappel de 85,7 % signifie que 14,3 % des données sensibles ont échappé au masquage.

Recherche hybride

Combinaison d'une recherche lexicale (fondée sur les termes) et d'une recherche dense (fondée sur le sens), dont les résultats sont fusionnés — ici par la méthode RRF.

Reproductibilité

Capacité à obtenir exactement le même résultat à partir des mêmes entrées, sur une autre machine et à une autre date. Elle suppose le versionnage conjoint du code, des données et des dépendances.

Chapitre C

Reproduction de l'environnement

L'intégralité de la pile d'observabilité décrite au chapitre 6 est démarrable par une commande unique, conformément au besoin BNF-10. Les huit services sont alors disponibles localement, Grafana étant le seul exposé au-delà de la boucle locale.

Service	Rôle	Exposition
Grafana	Consultation unifiée des trois signaux	Seul service exposable, protégé par authentification
Prometheus	Collecte et stockage des métriques	Boucle locale uniquement
Alertmanager	Routage des alertes	Boucle locale uniquement
Loki	Centralisation des journaux	Boucle locale uniquement
Promtail	Collecte des journaux	Réseau interne
Tempo	Stockage des traces	Boucle locale uniquement
Collecteur OTLP	Réception et expurgation de la télémétrie	Boucle locale uniquement
Exportateur de test	Génération de charge pour validation	Boucle locale uniquement

TABLE C.1 – Les huit services conteneurisés de la pile d'observabilité et leur niveau d'exposition réseau.

Décision de gouvernance

Pourquoi un seul service exposé. Prometheus, Alertmanager, Loki et Tempo ne disposent d'aucune authentification native. Sur un réseau partagé, leur exposition permettrait à tout hôte du segment de les interroger — et Loki donne accès au journal d'audit. Seul Grafana, qui porte une authentification, a vocation à être joignable de l'extérieur.